# Linux Job Scheduler and Task Automation System

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

## BACHELOR OF TECHNOLOGY

**In**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**By**


| | |
|---|---|
| MONISHA RAGHINI CHELLE | 2520030537 |
| SHAIK SABIYA TABASSUM | 2520030502 |
| K. SIRI CHANDANA | 2520030453 |


Under the Esteemed Guidance of


**Dr. K. Anusha**
**Assistant Professor**
**Department of Computer Science and Engineering**






**Koneru Lakshmaiah Education Foundation**
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University**
**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**
**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL**




# Declaration

The Project Report entitled **"Linux Job Scheduler and Task Automation System"** is a record of Bonafide work of **MONISHA RAGHINI CHELLE - 2520030537, SHAIK SABIYA TABASSUM- 2520030502, K. SIRI CHANDANA - 2520030453** submitted in
partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

**MONISHA RAGHINI CHELLE 2520030537**
**SHAIK SABIYA TABASSUM 2520030502**
**K. SIRI CHANDANA 2520030453**

## K L (Deemed to be) University

### DEPARTMENT OF COMPUTER SCIENCE ENGINEERING




# CERTIFICATE

This is to certify that the mini project-based report entitled **"Linux Job Scheduler and Task Automation System"** is a bonafide work done and submitted by **MONISHA RAGHINI CHELLE - 2520030537, SHAIK SABIYA TABASSUM- 2520030502, K. SIRI CHANDANA – 2520030453** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

**Signature of the Guide**

**Signature of the Course Coordinator** **Signature of the HOD**

# ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide Dr. K. Anusha**,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

**MONISHA RAGHINI CHELLE 2520030537**
**SHAIK SABIYA TABASSUM 2520030502**
**K. SIRI CHANDANA 2520030453**

# Abstract

This project implements a terminal-based, one-time Linux job scheduler in C. A user selects an approved program, optionally enters an echo message, and supplies a future local date and time. The application validates the request, assigns a job ID, persists the job, monitors due times while the foreground application is open, launches the approved program through Linux process-control interfaces, captures its output and exit result, and records status and logs. The implementation uses process creation and execution, an anonymous pipe for result communication, file I/O for persistence, and a POSIX worker thread for each due job. The supplied verification report records a strict build and automated tests passing; however, documentation link reconciliation and separate repository upload remain unverified. The scheduler is foreground-only and does not promise execution after the application is closed.

Keywords: Linux, C, job scheduling, process control, inter-process communication, file I/O, POSIX threads

# Table Of Contents

ACKNOWLEDGEMENT ..... 4
Abstract ..... 5
CHAPTER - 1 INTRODUCTION ..... 12
1.1 Background of the Project ..... 12
1.2Problem Statement ..... 12
1.3 Scope of the Project ..... 13
1.4 Objectives of the Project ..... 13
1.5 Importance of the Application ..... 14
1.6 Target Users / Audience ..... 15
1.7 Motivation of the Project ..... 15
1.8 Need for the Project ..... 16
1.9 Proposed Solution ..... 16
1.10 Project Workflow ..... 17
1.11 Organization of the Project ..... 20
1.12 Expected Outcome ..... 20
CHAPTER - 2 OS CONCEPTS AND BACKGROUND STUDY ..... 22
2.1 Operating System Concepts Related to the Project ..... 22
2.1.1 Operating System Services ..... 22
2.1.2 Process Management ..... 23
2.1.3 Signals and Asynchronous Notification ..... 23
2.1.4 Dynamic Memory Management ..... 24
2.1.5 File Input and Output ..... 24
2.1.6 Threads and Synchronization ..... 24
2.1.7 Integration of Operating System Concepts ..... 25
2.2 Existing Methods / Algorithms ..... 25
2.2.1 Comparison of Polling and Timer-Based Operation ..... 26
2.3 Proposed Approach ..... 26
2.3.2 Scheduled-Time Calculation ..... 27
2.3.3 Process Execution and Result Communication ..... 27
2.3.4 Job Status and Cancellation ..... 28
2.3.5 Resource Cleanup ..... 28
2.4 Functional Requirements ..... 29
2.5 Non-functional Requirements ..... 29

2.6 Hardware and Software Requirments ..... 30
2.7 Constraints ..... 30
2.8 Advantages of the Proposed Approach ..... 31
2.8.1 Simple Terminal-Based Interface ..... 31
2.8.2 Separation of Responsibilities ..... 31
2.8.3 Actual Exection Results ..... 31
2.8.4 Persistent Job Information ..... 32
2.8.5 Separation of Responsibilities ..... 32
2.8.6 Practical OS Demonstration ..... 32
2.9 Background Study of Linux and POSIX Environment ..... 32
2.10 Relationship Between the Concepts ..... 33
2.11 Summary of the Background Study ..... 33
CHAPTER - 3 SYSTEM REQUIREMENTS ..... 35
3.1 Hardware Requirements ..... 35
3.2 Software Requirements ..... 35
3.2.1 Windows ..... 36
3.2.2 Ubuntu through WSL ..... 36
3.2.3 Visual Studio Code ..... 36
3.2.4 GCC ..... 37
3.2.5 Git ..... 37
3.3 Operating System Environment – Ubuntu / WSL ..... 37
3.3.1 Role of WSL in the Project ..... 38
3.3.2 Terminal-Based Execution ..... 38
3.3.3 Compilation and Execution Environment ..... 39
3.4 Development Tools and Compilers ..... 39
3.4.1 Source-Code Editor ..... 39
3.4.2 Compiler ..... 40
3.4.3 Project Compilation Structure ..... 40
3.4.4 Development Workflow ..... 41
CHAPTER - 4 TECHNOLOGY AND TOOLS USED ..... 42
4.1 C Programming ..... 42
4.2 GCC Compiler ..... 42
4.3 Linux System Calls and APIs ..... 43
4.3.1 Process Identification APIs ..... 43
4.3.2 Operating-System Information ..... 43
4.3.3 Process Creation and Control ..... 44
4.3.4 Alarm and Signal APIs ..... 44

4.3.5 File APIs ..... 45
4.3.6 Thread APIs ..... 45
4.4 POSIX Threads / Synchronization ..... 45
4.4.1 Monitor Thread ..... 46
4.4.2 Mutex ..... 46
4.4.4 Thread Completion ..... 47
4.5 Linux Commands and Utilities ..... 47
4.5.1 wsl ..... 47
4.5.2 cd ..... 48
4.5.3 ls ..... 48
4.5.4 gcc ..... 48
4.5.5 Running the Scheduler ..... 48
4.5.6 Relationship Between Commands ..... 49
4.6 Other Tools / Libraries Used ..... 49
4.6.1 Overall Technology Stack ..... 50
4.6.2 Summary ..... 50
CHAPTER - 5 SYSTEM ARCHITECTURE AND DESIGN ..... 51
5.1 Overall System Architecture ..... 51
5.2 System Workflow ..... 52
5.3 Flowchart / Data Flow ..... 53
5.4 Module Description ..... 55
5.4.1 Main Module — main.c ..... 55
5.4.2 Menu Module — menu.c ..... 55
5.4.3 Job Manager — job_manager.c ..... 55
5.4.4 Scheduler Module — scheduler.c ..... 55
5.4.5 Process Manager — process_manager.c ..... 55
5.4.6 Job Storage Module — job_storage.c ..... 56
5.4.7 Interaction Between Modules ..... 56
5.5 Input and Output Design ..... 56
5.5.1 Input Design ..... 56
5.5.2 Output Design ..... 57
5.5.3 Job Status Display ..... 57
5.5.4 Log File Design ..... 58
5.5.5 Cancellation and Error Messages ..... 58
5.5.6 Input and Output Summary ..... 58
CHAPTER - 6 IMPLEMENTATION ..... 59
6.1 Module-wise Implementation ..... 59

6.1.1 main.c....59
6.1.2 menu.c....59
6.1.3 job_manager.c....59
6.1.4 scheduler.c....59
6.1.5 process_manager.c....59
6.1.6 job_storage.c....59
6.2 Algorithm / Pseudocode....59
6.2.1 Algorithm for Scheduling and Executing a Job....60
6.2.2 Pseudocode....60
6.3 Core C Implementation....62
6.3.1 Data Structures....62
6.3.2 Input Validation....63
6.3.3 Job Management and Status Handling....63
6.3.4 Error Handling....63
6.4 Linux System Calls / APIs Used....63
6.4.1 fork()....63
6.4.2 exec() Family....64
6.4.3 waitpid()....64
6.4.4 Pipe APIs....64
6.4.5 File APIs....64
6.4.6 POSIX Thread APIs....64
6.5 Process / Thread Implementation....65
6.5.1 Process Creation....65
6.5.2 Exit-Status Collection....65
6.5.3 Worker-Thread Lifecycle....65
6.6.1 Anonymous Pipe Communication....66
6.6.2 Thread Completion and Coordination....66
6.6.3 Cancellation and Dispatch....66
6.7 Memory Management Implementation....66
6.7.1 Input and Buffer Management....67
6.7.2 Resource Cleanup....67
6.7.3 Separate Memory-Management Demonstration....67
6.7.4 Summary....67
CHAPTER - 7 EXECUTION AND DEMONSTRATION....68
7.1 Compilation in Ubuntu....68
7.2 Running the Application....68
7.3 Sample Inputs and Outputs....68

7.4 Task Execution and Logs....69
7.5 Input Validation and Error Handling....70
7.6 Cancellation of Pending Jobs....71
7.7 Failure Handling....72
7.8 Demonstration of Operating System Concepts....73
7.9 Known Limitations....74
7.10 Summary....74
CHAPTER - 8 CONCLUSION....75
8.1 Summary of the Project....75
8.2 Objectives Achieved....75
8.2.1 Terminal-Based User Interface....75
8.2.2 Task Scheduling....75
8.2.3 Integration of Operating System Concepts (CO1–CO6)....75
8.2.4 Process Management....75
8.2.5 Dynamic Memory Management....76
8.2.6 File Storage....76
8.2.7 Thread Management....76
8.3 Results Achieved....76
8.4 Skills and Learning Outcomes....76
8.4.1 Linux Environment and Development Tools....76
8.4.2 C Modular Programming....77
8.4.3 System Calls and APIs....77
8.4.4 Time-Based Scheduling....77
8.4.5 Process Management....77
8.4.6 Memory and File Management....77
8.4.7 Thread Management and Synchronization....77
8.5 Overall Conclusion....77
CHAPTER - 9 REFERENCES....78
9.1 Books....78
9.2 Research Papers....78
9.3 Linux / GNU Documentation....78
9.3.1 GCC Documentation....78
9.3.2 Linux Process Documentation....78
9.3.3 Linux Signal and Alarm Documentation....78
9.3.4 Linux File I/O Documentation....78
9.3.5 POSIX Thread Documentation....78
9.4 Project Resources....79

9.4.1 Project Source Repository .....79
9.4.2 Project README and Source Files .....79
9.4.3 Project Execution Screenshots.....79
9.5 Reference Usage in the Project.....79
9.6 Summary of References.....79

# CHAPTER - 1 INTRODUCTION

## 1.1 Background of the Project

An operating system is an essential component of a computer system that manages hardware resources and provides services to application programs. It acts as an interface between users, applications, and computer hardware. Major operating-system responsibilities include process management, memory management, file management, input/output operations, security, and resource allocation. Linux is widely used in academic, development, and server environments because it provides powerful command-line utilities and system interfaces for performing these operations.

In many situations, users need to execute a program at a particular time rather than run it immediately. For example, a user may want to execute a program after a specified delay, generate a report at a particular time, or run a routine system task without manually starting it. Executing such tasks manually requires the user to remember the required time and initiate the program. A scheduling system can reduce this manual effort by accepting a task and its execution time in advance.

Linux provides several mechanisms for process execution and task automation. Operating-system interfaces allow programs to create processes, execute other programs, wait for their completion, and inspect their exit status. File operations allow applications to preserve information, while inter-process communication mechanisms allow related processes to exchange data. These capabilities provide the foundation for implementing a simple task scheduler.

The **Linux Job Scheduler and Task Automation System** is a terminal-based project developed in the C programming language. It allows users to select an approved program, specify a future date and time, and schedule the program for automatic execution. The application maintains job records and provides options to view scheduled jobs, inspect their status and logs, and cancel pending tasks.

The project focuses on demonstrating selected Operating Systems and Systems Programming concepts through an understandable working application. It connects scheduling logic with process control, file input/output, inter-process communication, input validation, and POSIX thread management. The implementation is intended as an educational prototype rather than a replacement for established Linux scheduling services.

## 1.2Problem Statement

Executing programs at specific times often requires users to start them manually or configure an existing scheduling utility. For beginners studying operating systems, it can also be difficult to understand how scheduled tasks connect to process creation, execution, termination, and result reporting.

The problem addressed by this project is **to develop a system that allows users to schedule programs or tasks for automatic execution at specified times**. The system should accept a program selection and a valid future execution time, retain the job information, and execute the selected program when it becomes due while the scheduler is running.

The system must also provide a way to determine whether a scheduled task is waiting, has completed successfully, has failed, or has been cancelled. Merely accepting a task and displaying a confirmation message is insufficient; the application must execute the selected program and record its actual result.

Several challenges must be considered while developing the system. The application must validate date and time input, prevent unsupported programs from being executed, preserve job records, handle process-creation or execution errors, and capture the correct exit status. It must also ensure that a cancelled pending task is not subsequently launched.
Therefore, the project aims to provide a simple and safe terminal interface for scheduling approved programs, monitoring their execution, and examining the results through job records and log files.

## 1.3 Scope of the Project

The scope of this project is limited to a Linux-based, terminal-operated job scheduling application implemented in C. It concentrates on one-time scheduled execution and on demonstrating selected operating-system interfaces.
The system supports four approved programs:

- **date:** Displays the actual date and time when the program executes.
- **echo:** Displays a user-provided message when the scheduled task executes.
- **true:** Provides a successful execution case with exit code 0.
- **false:** Provides a failure-status test case with exit code 1.

These programs are useful for testing different execution outcomes. The echo program demonstrates the handling of user-provided arguments, while true and false help verify that the scheduler distinguishes successful execution from a nonzero exit status.
The main features within the project scope include:

1. Scheduling a selected program for a future date and time.
2. Validating scheduling input and job records.
3. Assigning job identifiers and maintaining job information.
4. Checking scheduled times and dispatching due tasks.
5. Creating processes and collecting their execution results.
6. Recording program output, exit codes, status information, and logs.
7. Displaying scheduled jobs and their execution details.
8. Cancelling jobs that are still pending.
9. Preserving job records across application restarts.

The application uses a command-line interface, so a graphical user interface or web interface is not included. It also uses an allowlist of supported programs instead of accepting arbitrary shell commands.
An important limitation is that the scheduler operates in the foreground. It must remain running to monitor and execute jobs at their scheduled times. It is not implemented as a continuously running background daemon, and users should not assume that pending jobs will execute while the application is closed. The system is also not intended to provide all the advanced scheduling policies available in production-grade scheduling tools.

## 1.4 Objectives of the Project

The primary objective is to develop a functional Linux job scheduler that automatically executes approved programs at user-specified future times. The project also aims to demonstrate how

operating-system facilities support program execution and result management.
The specific objectives are:

1. **To provide a terminal-based interface:** Allow users to schedule tasks, view scheduled jobs, inspect execution results, cancel pending tasks, and exit the application.
2. **To validate scheduling information:** Check the date and time entered by the user and reject invalid or unsuitable scheduling requests.
3. **To maintain job records:** Assign unique job identifiers and store information required to identify and monitor scheduled tasks.
4. **To execute programs automatically:** Detect when a job becomes due and launch its approved program without requiring the user to start it manually at that moment.
5. **To demonstrate process control:** Use Linux process-management interfaces to create and execute processes, wait for completion, and collect exit information.
6. **To capture execution results:** Record actual program output and exit status so that successful and failed executions can be distinguished.
7. **To support job monitoring:** Provide options for listing jobs and viewing their status and logs.
8. **To support cancellation:** Allow users to cancel a pending task and prevent that task from executing after cancellation.
9. **To demonstrate file handling and IPC:** Use persistent job storage, log files, and the implemented inter-process communication mechanism to manage and report execution results.
10. **To verify correctness:** Test normal execution, failed execution, invalid inputs, cancellation, persistence, and relevant error-handling cases.

These objectives connect the functional requirements of the project with the Operating Systems and Systems Programming concepts studied in the course.

## 1.5 Importance of the Application

Automatic task execution is useful because it reduces the need for users to remember and manually start every program at the required time. A scheduling application can accept the task information in advance and monitor when execution should begin.

The project is particularly important from an educational perspective. A simple terminal scheduler makes several operating-system concepts visible in a single application. Students can observe the difference between scheduling a job and actually executing it, as well as the difference between a program's exit status and the scheduler's own operation.

The application also demonstrates why result tracking is important. A scheduled task may fail because the process could not be created, the program could not be executed, or the program returned a nonzero exit status. Capturing the output and result helps the user understand what happened instead of relying only on a scheduling confirmation.

The project demonstrates the use of file operations to retain job information, process-control interfaces to launch and monitor programs, and an anonymous pipe to communicate execution results. The worker-thread implementation also provides an opportunity to examine thread creation and lifecycle management.

Another important aspect is safe execution. Restricting the scheduler to a small set of approved programs reduces the risks associated with executing arbitrary user-provided commands. The user

can supply a message for echo, but that message is treated as program data rather than as an arbitrary shell command.
Overall, the application combines practical task automation with concepts from operating systems, making it suitable for demonstration, experimentation, and introductory systems-programming education.

## 1.6 Target Users / Audience

The primary audience for the Linux Job Scheduler and Task Automation System consists of students studying Operating Systems and Systems Programming. The project provides a practical example of how concepts discussed in theory can be implemented in a C application running on Linux.
Students can use the application to understand how scheduled jobs are represented, how a program is launched at a specified time, and how the result of execution is collected. They can also study how job information is stored and how the application handles pending tasks, successful executions, failures, and cancellation.
The project may also be useful to:

- **Programming learners:** Students learning C programming and Linux system interfaces.
- **Operating-system learners:** Students exploring process creation, process termination, file operations, and IPC.
- **Linux users:** Users interested in understanding basic task-scheduling behavior through a small prototype.
- **Project evaluators:** Faculty members assessing the practical application of Operating Systems and Systems Programming concepts.

The application is designed for a controlled educational environment. It does not claim to offer the reliability, security controls, or comprehensive scheduling features required for production servers or critical business operations.

## 1.7 Motivation of the Project

The motivation for developing this project comes from the need to understand operating-system concepts through practical implementation rather than theoretical explanation alone. Topics such as process creation, execution, waiting, file handling, and inter-process communication can be difficult to visualize when studied only through definitions.

A task scheduler provides a natural example in which these concepts work together. The user first provides a task and a future execution time. The application stores the request, monitors the time, launches the program when it becomes due, and records the result. Each stage illustrates a different responsibility within the application.

The project also provides an opportunity to work with C and Linux in a structured way. Instead of writing one large program containing all functionality, the implementation separates responsibilities across modules for menu interaction, job management, scheduling, process execution, and storage. This modular approach makes the source code easier to inspect, test, and explain.

The choice of a terminal-based interface is intentional because it keeps the project focused on operating-system behavior rather than graphical design. The four supported programs provide simple test cases without requiring external services or complicated setup.
Through this project, the team aims to connect classroom learning with a functioning prototype and develop a clearer understanding of how a user-level application coordinates with the operating system.

## 1.8 Need for the Project

There is a need for a scheduling system that can accept a task in advance and initiate execution when its specified time arrives. Manual execution depends on user availability and attention. It may be inconvenient when the same type of task needs to be started at a particular time.
A scheduling application also needs to maintain a clear record of what has been requested and what has happened. Without job status and logs, users may find it difficult to distinguish a task that is still waiting from one that has already executed or failed.
From an academic perspective, the project is needed to demonstrate how a real application uses operating-system facilities. Process creation, program execution, waiting for child processes, communication through pipes, and file-based persistence can be studied together through one application.
Input validation and failure handling are equally important. A scheduling system should reject invalid times and unsupported program choices rather than blindly accepting every request. It should also record actual execution outcomes rather than marking every completed attempt as successful.
The project addresses these needs within a manageable scope. It provides a terminal menu for scheduling and monitoring tasks, a limited set of approved programs, persistent job records, and result logging. Its foreground-only behavior is documented so that the prototype's capabilities are not confused with those of a background scheduling service.

## 1.9 Proposed Solution

The proposed solution is a modular, terminal-based Linux application written in C. It accepts task information from the user, validates the request, stores the job, monitors its scheduled time, executes the selected approved program, and records the result.
The user begins by selecting the option to schedule a task. The application displays the available programs: date, echo, true, and false. If echo is selected, the user also provides a message to be displayed when the task runs. The user then enters a future date and time in the required format.
After validating the input, the application assigns a job ID and records the job. The scheduler checks whether the task has become due while the application is running. When the scheduled time arrives, the process-management module launches the selected program using the implemented Linux process-control mechanisms.
The application collects the program's output and execution result. The exit code and status are used to distinguish success from failure. The job record and associated log allow the user to inspect the result later through the menu.
The application provides the following main menu operations:

1. Schedule a Task.
2. View Scheduled Jobs.
3. View Job Status and Logs.
4. Cancel a Pending Task.
5. Exit.

The proposed solution separates menu interaction, job management, scheduling, process execution, and file storage into distinct modules. This separation makes the program easier to maintain and allows the team to explain how each module contributes to the overall workflow.
The solution is intended as an educational prototype. It demonstrates selected OS facilities but does not implement a complete production-grade scheduler or guarantee task execution while the foreground application is closed.

## 1.10 Project Workflow

The project follows a sequence of steps beginning with user input and ending with execution-result reporting.

**Step 1: Application startup.** The user starts the Linux Job Scheduler application from the terminal. The application initializes its job-management components and loads the available stored job information.

**Step 2: Task selection.** The user chooses the scheduling option and selects one of the approved programs: date, echo, true, or false. If the user selects echo, the application also collects the message to be displayed.

**Step 3: Date and time input.** The user enters the intended execution date and time. The application checks the input format and validates whether the time is suitable for scheduling.

**Step 4: Job creation and storage.** If the request is valid, the application creates a job record, assigns an ID, sets the initial status to pending, and saves the job information.

**Step 5: Scheduled-time monitoring.** While the application remains open, the scheduler checks whether a pending job has reached its execution time. Jobs that are not yet due remain pending.

**Step 6: Process execution.** When a job becomes due, the process-management module launches the approved program. The application waits for the process to complete and obtains the actual execution result.

**Step 7: Output and result collection.** The program's output and exit information are captured through the implemented output-handling and IPC mechanisms. The application records the result in the job information and associated log.

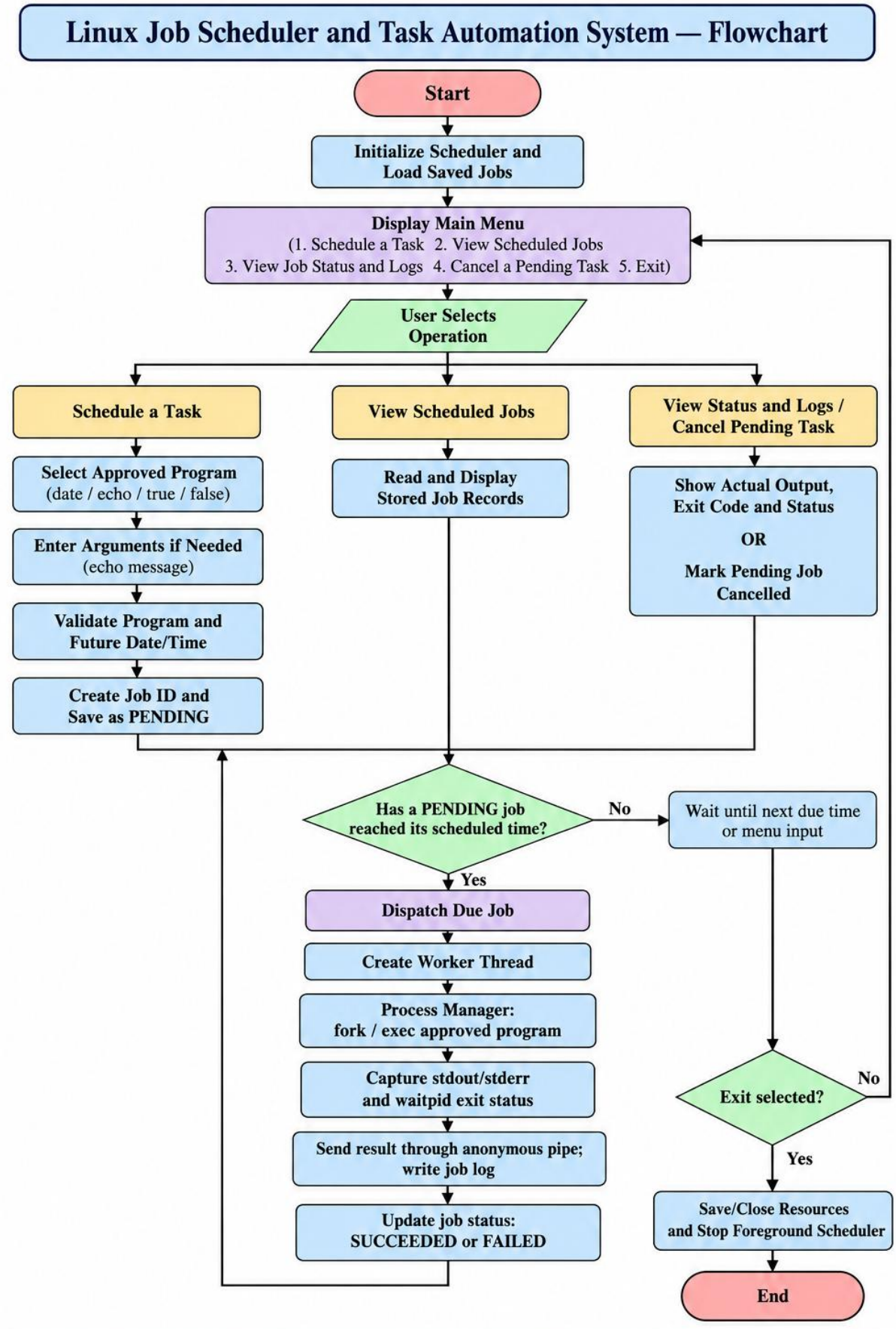
**Step 8: Status update.** The job is marked according to the actual outcome. A successful execution is recorded as succeeded, while a nonzero program exit status is recorded as failed. A pending job cancelled before execution is recorded as cancelled.

**Step 9: Result inspection.** The user can view the job list and inspect individual job status and logs using the menu options.

**Step 10: Application exit.** The user selects the exit option to stop the application. Because the scheduler runs in the foreground, it no longer monitors pending jobs after exit.

The overall workflow can be summarized as:
**User Input → Validation → Job Storage → Time Monitoring → Process Execution → Output and Exit-Code Collection → Status Update → Logs and Results**

This workflow shows how the application combines scheduling logic with selected operating-system services to execute and monitor tasks.


Linux Job Scheduler and Task Automation System — Flowchart
Start
Initialize Scheduler and Load Saved Jobs
Display Main Menu
(1. Schedule a Task 2. View Scheduled Jobs
3. View Job Status and Logs 4. Cancel a Pending Task 5. Exit)
User Selects Operation
Schedule a Task
View Scheduled Jobs
View Status and Logs / Cancel Pending Task
Select Approved Program
(date / echo / true / false)
Enter Arguments if Needed
(echo message)
Validate Program and Future Date/Time
Create Job ID and Save as PENDING
Read and Display Stored Job Records
Show Actual Output, Exit Code and Status
OR
Mark Pending Job Cancelled
Has a PENDING job reached its scheduled time?
No
Wait until next due time or menu input
Yes
Dispatch Due Job
Create Worker Thread
Process Manager: fork / exec approved program
Capture stdout/stderr and waitpid exit status
Send result through anonymous pipe; write job log
Update job status: SUCCEEDED or FAILED
Exit selected?
No
Yes
Save/Close Resources and Stop Foreground Scheduler
End
The scheduler must remain running; this prototype is not a background daemon.

## 1.11 Organization of the Project

The project is organized into six chapters to present the problem, requirements, design, implementation, testing, and conclusions in a systematic manner.

**Chapter 1 — Introduction:** Introduces the project background, problem statement, scope, objectives, importance, target users, motivation, and expected outcomes.

**Chapter 2 — OS Concepts and Background:** Explains the operating system concepts relevant to the project, including process management, inter-process communication, file handling, and concurrency.

**Chapter 3 — System Requirements:** Describes the functional and non-functional requirements, hardware and software requirements, development environment, tools, and application constraints.

**Chapter 4 — Technology and Tools:** Presents the programming language, compiler, build tools, Linux environment, APIs, and other technologies used in the project.

**Chapter 5 — System Architecture and Design:** Explains the system architecture, module relationships, job workflow, scheduling process, process-execution flow, storage design, and error-handling approach.

**Chapter 6 — Implementation:** Describes the source files and functions used to implement the terminal menu, job management, scheduler, process management, and storage modules.

**Chapter 7 — Execution and Demonstration:** Presents the compilation and execution process, sample scheduling operations, execution results, job-status updates, logs, and screenshots.

**Chapter 8 — Conclusion:** Summarizes the project implementation, operating system concepts demonstrated, achievements, and limitations.

**Chapter 9 — References:** Lists the books, official documentation, educational resources, and project repository consulted during development.

.

## 1.12 Expected Outcome

The expected outcome of the project is a functional terminal-based application that allows users to schedule approved programs for automatic execution at specified future times while the scheduler is running.

The application is expected to validate user input, assign job IDs, retain job information, detect due tasks, launch the selected programs, and record their actual output and exit status. Users should be able to list scheduled jobs, inspect their logs, and cancel pending tasks through the terminal menu.

The four supported programs provide different ways to evaluate the scheduler. The date program demonstrates output generated at execution time. The echo program demonstrates the use of a

user-supplied message. The true program provides a successful execution case, while false provides a nonzero-exit case that should be identified as a failed job.

From an Operating Systems and Systems Programming perspective, the project is expected to demonstrate selected process-control operations, anonymous-pipe communication, file-based job persistence, record validation, and POSIX worker-thread lifecycle management. The exact course-outcome coverage must be assessed against the implementation and the official course requirements; the project does not implement every topic in all six course modules.

The project is also expected to provide a foundation for further learning about scheduling policies, background services, synchronization, error recovery, and more advanced concurrency.

These are potential improvements rather than claims about features already implemented.
In conclusion, the expected result is a small but practical Linux scheduling prototype that combines task scheduling, program execution, status tracking, and result logging in an educational environment.

# CHAPTER - 2 OS CONCEPTS AND BACKGROUND STUDY

## 2.1 Operating System Concepts Related to the Project

An operating system provides the environment in which application programs execute. It manages system resources and exposes interfaces that applications can use to create processes, access files, communicate with other processes, and perform input/output operations. The Linux Job Scheduler and Task Automation System uses selected operating-system facilities to schedule and execute programs at specified times.

The project combines application-level scheduling logic with Linux process management and file operations. The scheduler determines when a task becomes due, while the process-management component is responsible for launching the selected program and collecting its execution result. The storage component maintains job records and logs so that users can inspect the outcome of scheduled tasks.

This section explains the operating-system concepts relevant to the implementation, including OS services, process management, signals, dynamic memory management, file input/output, threads, and the integration of these concepts.

### 2.1.1 Operating System Services

An operating system provides services that enable applications to use computer resources without directly controlling hardware. These services include program execution, file access, process management, memory allocation, and communication between processes.

In Linux, application programs can access many operating-system services through system calls and library interfaces. For example, a program can create a new process, execute another program, wait for process termination, or open and write to a file.

The Linux Job Scheduler uses these services to implement its scheduling workflow. The terminal interface accepts user input, the scheduler checks job execution times, and the process manager launches an approved program when its scheduled time arrives. File operations are used to preserve job information and execution logs.

The application is a user-space program. It requests operating-system services through the available Linux interfaces rather than implementing its own operating-system kernel. This distinction is important because the scheduler decides when a task should run, while Linux provides the mechanisms needed to execute and manage the process.

### 2.1.2 Process Management

A process is a program in execution. It has an execution context and operating-system-managed resources, such as a process identifier, memory space, and open file descriptors. Process management includes creating processes, executing programs, waiting for completion, and handling termination status.

The project uses process-management mechanisms to execute scheduled programs. When a job becomes due, the process manager launches the selected approved program. Depending on the implementation, this involves interfaces such as fork(), an exec() family function, and waitpid().

The fork() system call creates a child process. An exec() family function replaces the current process image with the requested program. The parent can use waitpid() to wait for the child process and collect information about its termination.

These mechanisms allow the scheduler to execute programs such as date, echo, true, and false. The program's actual exit status is important when determining the job result. A successful exit normally returns status 0, whereas false returns a nonzero exit status and provides a useful test case for failure handling.

The scheduler records the execution result so that users can distinguish a successful job from a failed one. Process management therefore forms a central part of the proposed application.

### 2.1.3 Signals and Asynchronous Notification

Signals are software notifications used in Linux and other Unix-like operating systems to notify a process that an event has occurred. Examples include SIGINT, commonly generated by pressing Ctrl+C in a terminal, and SIGCHLD, which can notify a process about changes in a child process's state.

Signals are useful when a program must respond to events without continuously checking every possible condition. For example, an application may use signal handling to initiate controlled shutdown or to respond to a child-process event.

In a task scheduler, signals could be used to support notifications or shutdown handling. However, signal-based scheduling is not the same as scheduling a job for a future date and time. The application must still determine when the job is due and then initiate execution.

The current project should claim signal handling only if a signal handler or other signal-related mechanism is actually implemented in the source code. If the scheduler relies on time checking, process waiting, and pipes without using signals, signals should be described as a related operating-system concept rather than an implemented feature.

### 2.1.4 Dynamic Memory Management

Dynamic memory management allows a program to request memory while it is running and release that memory when it is no longer required. In C, functions such as malloc(), calloc(), realloc(), and free() are commonly used for dynamic allocation and deallocation.

A job scheduler may need memory to hold job records, program arguments, log information, or temporary data structures. The implementation must ensure that the allocated memory is large enough for the information being stored and that memory is released when it is no longer needed. Incorrect memory handling can cause problems such as memory leaks, invalid memory access, buffer overflows, and use-after-free errors. Input validation and appropriate bounds checking help prevent these problems.

Memory management is particularly relevant when the application loads saved job records or creates temporary copies of job information for execution. If dynamically allocated memory is used, the program should release it along every relevant execution path, including error paths.

Memory-safety testing can help identify certain memory errors in the application. However, passing a memory diagnostic test does not mean that the project implements virtual-memory management, paging, or address translation. Those are separate operating-system topics and should not be claimed unless they are genuinely implemented or demonstrated.

### 2.1.5 File Input and Output

File input/output allows an application to store information beyond the lifetime of a single function call or program execution. Persistent storage is important for a scheduler because users need to inspect previously scheduled jobs and their results.

The project stores job information in a database file and records execution output in per-job log files. The stored job information can include the job ID, scheduled time, selected program, optional arguments, status, and execution result, according to the implemented record format.

When the application starts, it can load previously saved records. When a new job is scheduled or its status changes, the storage component updates the corresponding information. Log files provide additional details about the output generated by the executed program.

Reliable file handling requires checking whether files can be opened, read, and written successfully. The application should also handle malformed records and storage failures without silently reporting success.

Persistence has an important limitation: saving a job record does not by itself guarantee that a job will execute while the application is closed. In this project, scheduling continues only while the foreground scheduler is running. The stored records help preserve job information and support the implemented restart behavior.

### 2.1.6 Threads and Synchronization

A thread is an execution path within a process. Threads in the same process share many resources, including the process's address space and global data. POSIX threads, commonly called pthreads, provide an interface for creating and managing threads in Linux applications.

A scheduler may use a worker thread to carry out a job's execution workflow. This can help separate the work associated with executing a task from other application responsibilities.
Functions such as pthread_create() and pthread_join() are used to create a thread and wait for its termination.

The current implementation uses a worker-thread lifecycle for due jobs, according to the supplied project audit. The jobs are dispatched sequentially because the worker is joined before final status handling continues. Creating a thread does not automatically mean that multiple jobs execute concurrently.

Synchronization becomes important when multiple threads access shared data at the same time. Mutexes, condition variables, and semaphores can coordinate access and prevent race conditions. A race condition may occur when the result depends on the timing of operations performed by different threads.

The project should claim the use of mutexes, condition variables, or semaphores only if these mechanisms exist in the actual source code. The presence of a worker thread alone does not demonstrate all concurrency and synchronization concepts.

### 2.1.7 Integration of Operating System Concepts

The scheduler integrates several operating-system concepts into a single application. The user interacts with the terminal menu to submit a task. The job manager validates the request and maintains the job record. The scheduler monitors the execution time, while the process manager launches the selected program and collects its result. The storage module saves job information and execution logs.

The process-control mechanism is responsible for creating and executing the selected program. File input/output preserves information about the job and its outcome. An anonymous pipe can carry a structured execution result between related processes, while a worker thread can manage the execution workflow.

These components work together to fulfil the primary requirement of automatic task execution at specified times. The scheduler decides when a job is due; the operating system provides the process and file interfaces needed to execute the job and preserve its results.

The integration of these concepts also makes the project useful for academic demonstration. Students can examine how the modules interact rather than studying process management, file operations, and communication as completely separate topics.

## 2.2 Existing Methods / Algorithms

Task scheduling can be implemented using different approaches, depending on the required accuracy, complexity, and system behavior. Common approaches include repeatedly checking the current time, waiting until a timer expires, and using an established operating-system scheduling service.

Polling repeatedly checks whether a task has reached its scheduled time. Timer-based waiting allows a program to wait for a specified interval before checking again. Operating-system tools such as cron and systemd timers provide more established scheduling facilities for recurring or time-based execution.

The Linux Job Scheduler is an educational implementation that performs time monitoring within its own application. Its behavior should be described according to the actual scheduler code rather than assuming that it uses a particular timer or signal mechanism.

### 2.2.1 Comparison of Polling and Timer-Based Operation

**Polling approach:** In polling, the application checks the current time and compares it with the scheduled time of pending jobs. If a job is not due, the application waits for a short interval before checking again. Polling is relatively easy to understand and implement, but frequent checks can use unnecessary CPU time. A long polling interval can also introduce a delay between the requested execution time and the actual start time.

**Timer-based waiting:** A timer-based approach arranges for the program to wait until a specified interval has elapsed or until another event occurs. This can reduce repeated checks. The implementation must still account for newly scheduled jobs, cancellation requests, interruptions, and changes in the next due time.

**System scheduling services:** Linux utilities such as cron and systemd timers provide facilities for scheduling programs outside a small educational application. They are more suitable for many real-world automation requirements, but they do not necessarily expose the internal scheduling and process-management logic that students are expected to implement in this project.

The main difference is how the program waits for a task to become due. Both polling and timer-based waiting can be used in a scheduler, but their CPU usage, responsiveness, and implementation complexity differ.

For this project, the implementation must be inspected to determine the exact waiting mechanism. If it calculates the time until the next due job and waits for input or the next scheduling event, document that behavior. Do not state that the application uses SIGALRM unless the source code actually implements it.

## 2.3 Proposed Approach

The proposed approach is to provide a modular C application that accepts a scheduled task, validates the requested time, stores the job, checks when it becomes due, executes the approved program, and records the result.

The user interacts with the application through a terminal menu. The program selection and optional arguments are collected before the scheduled date and time are validated. A valid request receives a job ID and is saved with a pending status.

While the scheduler remains running, it monitors pending jobs. When a job reaches its scheduled

time, the scheduler dispatches it to the process-management component. The program's output and exit result are captured and saved, and the job status is updated. Users can subsequently view scheduled jobs, inspect logs, or cancel jobs that have not yet executed.

The approach emphasizes validation, safe execution, traceable results, and clear separation of responsibilities. It does not promise background execution after the application exits.

### 2.3.1 Task Input and Validation

The first stage is to collect the task details from the user. The application presents the available programs and allows the user to select one. If echo is selected, the user must also enter the message that should be displayed when the job executes.

The application then requests a scheduled date and time in the required format. Validation should check the format and calendar values, as well as whether the requested time is in the future. Invalid input should be rejected with a useful error message rather than being stored as a valid job.

The selected program must also be checked against the supported-program allowlist. This restricts execution to approved programs and prevents the scheduler from treating arbitrary user input as a shell command.

After validation succeeds, the job manager creates a record with a unique ID and a pending status. The record is then stored for later monitoring and execution.

### 2.3.2 Scheduled-Time Calculation

The scheduler must determine when each pending job is due. It compares the stored scheduled time with the current system time and identifies jobs whose execution time has arrived.
A simple time comparison can be expressed as:

- If the current time is earlier than the scheduled time, keep the job pending.
- If the current time is equal to or later than the scheduled time, consider the job due.
- If the job has been cancelled, do not dispatch it.

The implementation must handle invalid or unavailable time values appropriately. It should also avoid confusing the time entered by the user with the actual execution time. For example, a date task displays the current date and time when the program runs, not necessarily the exact time originally entered by the user.

The scheduler's waiting mechanism determines how often it checks for due work. Its behavior and timing precision depend on the implementation and operating-system scheduling.

### 2.3.3 Process Execution and Result Communication

When a job becomes due, the scheduler passes the job information to the process-management component. The component validates the selected program and launches it using the implemented Linux process-control interfaces.

The process manager waits for completion and determines the actual exit result. Where the

implementation uses an anonymous pipe, the pipe communicates a structured execution-result record between the related processes. The application can then use that result to update the job status.

The output of the program is captured in the associated log file. For the four supported programs, date produces the actual execution time, echo produces the saved user message, true returns exit code 0, and false returns exit code 1.

This approach allows the scheduler to record real results instead of assuming that every launched task succeeds. Process-creation failures and execution errors should also be reported accurately.

### 2.3.4 Job Status and Cancellation

When a job becomes due, the scheduler passes the job information to the process-management component. The component validates the selected program and launches it using the implemented Linux process-control interfaces.

The process manager waits for completion and determines the actual exit result. Where the implementation uses an anonymous pipe, the pipe communicates a structured execution-result record between the related processes. The application can then use that result to update the job status.

The output of the program is captured in the associated log file. For the four supported programs, date produces the actual execution time, echo produces the saved user message, true returns exit code 0, and false returns exit code 1.

This approach allows the scheduler to record real results instead of assuming that every launched task succeeds. Process-creation failures and execution errors should also be reported accurately.

### 2.3.5 Resource Cleanup

Resource cleanup is an important part of reliable systems programming. Processes, threads, file descriptors, allocated memory, and open files must be managed appropriately throughout the application's lifecycle.

The process manager must collect the termination status of child processes so that completed children do not remain unreaped. Pipes and file descriptors should be closed when they are no longer needed. Files should be closed after reading or writing, and dynamically allocated memory should be released if it is used.

Worker threads must also be managed according to the implementation. If a thread is joinable, the application should join it or otherwise manage its lifecycle correctly. The scheduler should save necessary job-state changes and close resources when the user selects Exit.

Because the current application is foreground-only, exiting stops its scheduling loop. Resource cleanup must not falsely imply that pending jobs will continue to execute in the background.

## 2.4 Functional Requirements

Functional requirements specify what the application must do in response to user actions and scheduled events.

| Requirement | Description |
|---|---|
| Schedule a task | Allow the user to select date, echo, true, or false and enter a future local date and time. |
| Custom echo message | Collect the user's message, save it with the job, and display and log it when the job executes. |
| Automatic dispatch | Monitor pending jobs and execute due jobs while the foreground scheduler is running. |
| Job identification | Assign a job ID and retain the information required to identify the scheduled task. |
| Job status | Record pending, succeeded, failed, or cancelled status according to the actual job lifecycle. |
| Output and logs | Capture actual program output and execution information in the associated job log. |
| View jobs | Display stored scheduled jobs and their current or final status. |
| View status and logs | Allow the user to inspect a selected job's result, exit code, and log output. |
| Cancellation | Allow a pending job to be cancelled and ensure that a cancelled job is not subsequently dispatched. |
| Persistence | Store job records so that information can be loaded again after a restart, subject to the implemented recovery behavior. |
| Safe execution | Restrict execution to approved programs and avoid constructing arbitrary shell commands from user input. |
| Input validation | Reject invalid date/time values, unsupported program selections, and malformed records. |
| Shutdown | Close resources and stop the foreground scheduler when the user exits. Pending tasks are not promised to execute while the application is closed. |

## 2.5 Non-functional Requirements

Non-functional requirements describe the qualities and constraints of the application rather than individual menu operations.

**Safety:** The scheduler should execute only approved programs. User-provided text, especially the custom echo message, should be handled as data rather than concatenated into an arbitrary shell command.

**Reliability:** Invalid input, malformed job records, process failures, and file-operation errors should be handled explicitly. The application should not report successful execution unless the recorded result supports that conclusion.

**Observability:** Users should be able to inspect job IDs, statuses, output, exit codes, and log files.

This makes it easier to understand the result of each scheduled task.

**Maintainability**: The source code is divided into modules for menu interaction, job management, scheduling, process execution, and storage. This separation makes individual responsibilities easier to understand and test.

**Portability:** The project targets Linux/POSIX behavior and is intended to run in Ubuntu on WSL2 or a suitable native Linux environment. Compilation and execution depend on the required compiler, build tools, and thread support being available.

**Resource management:** Processes, threads, file descriptors, and allocated memory should be managed carefully. Error paths must be considered as well as normal execution.

**Usability:** The menu should use clear option names and error messages. Date/time input requirements should be stated so that users can enter valid values without guessing.
**Scope transparency:** The application must clearly communicate that it is a foreground scheduler. It must not claim background execution, complete virtual-memory implementation, or synchronization mechanisms that are not present in the source.

## 2.6 Hardware and Software Requirments

The application does not require specialized hardware. It can run on a standard computer capable of running Linux or Ubuntu through Windows Subsystem for Linux (WSL2).

| Item | Requirement |
|---|---|
| **Hardware** | Standard computer capable of running Linux or WSL2; no special hardware is required. |
| **Operating system** | Ubuntu/Linux or a compatible POSIX environment. |
| **Programming language** | C. |
| **Compiler** | GCC or a compatible C compiler. |
| **Build tool** | GNU Make. |
| **Thread support** | POSIX pthread support, as required by the implementation. |
| **Interface** | Terminal or command-line interface. |
| **Storage** | Local file system for job records and log files. |
| **Optional testing tool** | Valgrind for memory diagnostics. |

The development workflow uses a terminal in Ubuntu/WSL and a source-code editor such as Visual Studio Code. GNU Make is used to compile the source files according to the project's Makefile.

The supplied audit reports a WSL2 Linux environment and GCC 15.2.0. These are audit-reported environment details, not universal minimum requirements. The exact compiler and operating-system versions should be confirmed on the machine used for the final demonstration.

## 2.7 Constraints

The scheduler has several implementation and operational constraints.

**Foreground execution:** The scheduler monitors and executes jobs while the application remains running. It is not a background daemon, so users must not assume that pending jobs will execute while the application is closed.

**Supported programs:** The current scope is limited to date, echo, true, and false. Arbitrary commands and unrestricted shell execution are outside the intended scope.

**One-time scheduling:** The project focuses on executing individual jobs at specified future times. Recurring schedules and advanced calendar rules are not included unless they are separately implemented.

**Concurrency limitations:** Although a POSIX worker-thread lifecycle is used, jobs are dispatched sequentially according to the supplied audit. The project should not claim parallel job execution or comprehensive synchronization support.

**Memory-management limitations:** Input validation and memory-safety testing do not amount to a full implementation of virtual memory, paging, or address translation.

**Storage limitations:** Job records and logs are stored locally. Inter-process database locking and guaranteed recovery from every power failure are not provided by the stated design.

**Execution timing:** The scheduler's actual start time may differ slightly from the requested time because of its waiting mechanism and operating-system scheduling. The project should report actual execution results rather than promise exact real-time execution.

These constraints define the current prototype's boundaries and provide a basis for identifying possible future improvements.

## 2.8 Advantages of the Proposed Approach

The proposed approach offers several advantages for an educational task-scheduling application.

### 2.8.1 Simple Terminal-Based Interface

The terminal menu provides a straightforward way to schedule tasks and inspect their results. A graphical interface is unnecessary for demonstrating the main scheduling and process-management concepts.

### 2.8.2 Separation of Responsibilities

The application separates user interaction, job management, scheduling, process execution, and file storage. This makes the source easier to understand, test, and maintain.

### 2.8.3 Actual Exection Results

The scheduler records the program's actual output and exit status. This allows the user to

distinguish successful execution from failure rather than relying on a generic completion message.

### 2.8.4 Persistent Job Information

Saving job records allows the application to retain scheduling information across restarts, subject to the recovery behavior implemented in the source.

### 2.8.5 Separation of Responsibilities

The limited allowlist of approved programs reduces the risks associated with executing arbitrary commands. The echo message is treated as program input rather than an unrestricted shell instruction.

### 2.8.6 Practical OS Demonstration

The project brings together selected concepts such as process control, anonymous-pipe communication, file input/output, input validation, and worker-thread management in a single application.
These advantages describe the educational and implementation benefits of the current approach; they do not imply that it provides every feature of a production scheduler.

## 2.9 Background Study of Linux and POSIX Environment

Linux provides system interfaces and command-line tools for creating and managing processes, accessing files, and coordinating execution. POSIX specifies a set of interfaces that support portability across compatible Unix-like systems.

The project is implemented in C and uses Linux/POSIX interfaces for the scheduling and execution workflow. Process-control functions allow the application to launch a selected program and collect its termination status. File operations support the storage of job records and execution logs. POSIX thread functions support the worker-thread lifecycle used by the implementation.

The terminal interface is appropriate for this project because it provides direct interaction with the application without requiring a separate graphical environment. The user enters task information through the menu and receives status information and results through terminal output and log files.

The development environment consists of a Linux or Ubuntu environment, a C compiler, GNU Make, and the required POSIX facilities. WSL2 can provide a Linux development environment on a Windows computer. The exact environment used for the final demonstration should be documented accurately.

The project does not implement a Linux kernel, replace system scheduling utilities, or create a background service. Instead, it demonstrates how a user-level application can use operating-system

interfaces to perform a limited scheduling task.

## 2.10 Relationship Between the Concepts

The operating-system concepts discussed in this chapter are related through the execution lifecycle of a scheduled job.

The process begins with user input. Input validation ensures that the program selection and scheduled time are suitable. The job manager creates a record and stores it through file operations. The scheduler monitors the due time and decides when execution should begin.

When the job becomes due, process management launches the selected program. The application waits for completion and collects the actual exit result. If an anonymous pipe is used, it carries the execution-result record between related processes. File input/output stores the job status and output for later inspection.

Thread management supports the worker-thread lifecycle used for executing due jobs. Resource-management practices ensure that processes are waited for, files and pipes are closed, and allocated memory is released where applicable.

These relationships show that task scheduling is not simply a timer operation. It requires coordination between time monitoring, process execution, result collection, storage, and user interaction.

The concepts also have different boundaries. A scheduler's time-monitoring mechanism is not the same as process control, and worker-thread creation alone does not demonstrate full synchronization. Similarly, file persistence does not guarantee background execution after the application exits. Maintaining these distinctions makes the project explanation technically accurate.

## 2.11 Summary of the Background Study

This chapter described the operating-system concepts that support the Linux Job Scheduler and Task Automation System. It covered OS services, process management, signals, dynamic memory management, file input/output, threads, and the integration of these concepts in the scheduling workflow.

The chapter also compared general scheduling approaches and described the proposed method of validating a task, saving its record, monitoring its due time, executing the approved program, and recording the actual result. Functional and non-functional requirements, hardware and software requirements, advantages, and implementation constraints were also discussed.

The project provides an educational example of how a C application can use Linux/POSIX facilities to automate selected tasks. Its implementation should be evaluated against the actual source code and test evidence. In particular, signal handling, synchronization primitives, and virtual-memory mechanisms should not be claimed as implemented unless the corresponding functionality is present and demonstrated.

The next chapter describes the system design, including the architecture, module interactions, data flow, and scheduling workflow.

# CHAPTER - 3 SYSTEM REQUIREMENTS

## 3.1 Hardware Requirements

The Linux Job Scheduler and Task Automation System is a software-based application developed using the C programming language. It does not require specialized hardware because the scheduling, process execution, file handling, and terminal interaction are performed using the facilities provided by the operating system.
A standard personal computer capable of running Windows with Windows Subsystem for Linux (WSL2), or a computer running a compatible Linux distribution, is sufficient for development and execution. The computer should have adequate memory and storage for the operating system, development tools, source files, job records, and execution logs.
The hardware requirements are described below.

| Component | Requirement |
|---|---|
| **Computer** | Standard desktop computer or laptop |
| **Processor** | Any processor supported by the selected operating system |
| **Memory** | Sufficient RAM to run the operating system, compiler, editor, and scheduler |
| **Storage** | Available disk space for source files, compiler tools, job records, logs, and documentation |
| **Input devices** | Keyboard and monitor for terminal interaction |
| **Network** | Required for downloading tools or accessing Git repositories; not normally required for local scheduling after setup |

No special graphics hardware, external device, or dedicated server is required. The application is designed to run as a terminal-based program.
The exact minimum RAM, processor, and disk-space values depend on the operating system and development environment. Therefore, this project does not specify an artificial hardware minimum that has not been tested.

## 3.2 Software Requirements

The software environment includes the operating system, development editor, C compiler, build utility, and version-control tool. These tools support source-code editing, compilation, execution, debugging, and project management.

The project uses Linux/POSIX functionality for process control, file operations, inter-process communication, and thread management. The required development environment must support the interfaces used by the source code.

| Software | Purpose |
|---|---|
| Windows | Host operating system used for development. |
| Ubuntu through WSL | Linux environment for compilation and execution. |
| VS Code | Source-code editing and project organization. |

| | |
|---|---|
| GCC | C compiler and linker. |
| Git | Version control and project repository management. |

### 3.2.1 Windows

Windows provides the host operating system for users who develop the project through Windows Subsystem for Linux. It supports the installation and operation of Visual Studio Code and provides access to development tools used to edit and manage the project.

Windows also supports the WSL feature, which allows a Linux distribution to run without requiring a separate physical Linux computer. The project files can be accessed through Visual Studio Code, while compilation and execution can be performed in the Linux environment.

In this project, Windows primarily acts as the host environment. The actual C compilation and execution should take place inside Ubuntu/WSL when the project is configured for that environment.

### 3.2.2 Ubuntu through WSL

Windows Subsystem for Linux (WSL) enables users to run a Linux environment on a Windows computer. Ubuntu can be installed as the Linux distribution used for developing and executing the project.

Ubuntu provides the Linux command-line environment and the system interfaces required by the application. Tools such as GCC and GNU Make can be installed inside Ubuntu and used to compile the C source files.

The project uses Linux/POSIX process and file interfaces, so running it inside Ubuntu provides an environment suitable for demonstrating these concepts. The terminal is also used to execute the compiled application, schedule jobs, view results, and inspect log files.

The supplied project audit reports a WSL2 Linux environment with kernel version 6.6.87.2. This value should be treated as the reported audit environment rather than a requirement for every installation. The actual version on the demonstration computer can be checked from the Ubuntu terminal.

### 3.2.3 Visual Studio Code

Visual Studio Code is used as the source-code editor for the project. It provides a convenient environment for opening the project directory, editing C files and header files, viewing the Makefile, and inspecting project documentation.

The editor also provides an integrated terminal, allowing the developer to run Linux commands and build the application without repeatedly switching between windows. When connected to Ubuntu through WSL, Visual Studio Code can open the project in the Linux development environment.

The project consists of multiple source files, so the editor's file explorer helps organize the menu, job-management, scheduler, process-management, and storage modules. This arrangement

supports modular development and makes it easier to locate errors.

Visual Studio Code is a development tool rather than the compiler itself. The actual compilation is performed by GCC in the configured Linux environment.

### 3.2.4 GCC

The GNU Compiler Collection (GCC) is used to compile the C source files into an executable program. It translates the source code into machine code and reports compilation errors or warnings that may need to be corrected.

The project contains multiple C files and corresponding header files. GCC compiles the implementation and links the required components into a single executable. The project Makefile defines the build commands and compilation dependencies.

A typical development workflow uses the following commands from the Ubuntu terminal:

```
gcc --version
make
```

The first command displays the installed GCC version. The second invokes the project's Makefile to build the application. The exact command depends on the targets defined in the supplied Makefile.

If the implementation uses POSIX threads, the build configuration must also include the appropriate pthread compiler and linker options. The project should be considered successfully compiled only after the actual build completes without errors.

The supplied audit reports GCC 15.2.0. This is an audit-reported version and should be replaced with the version displayed by the final demonstration environment if it differs.

### 3.2.5 Git

Git is a distributed version-control system used to maintain a history of source-code changes. It allows developers to track modifications, create commits, and restore earlier versions of files when necessary.

For this project, Git can be used to maintain the source code, header files, Makefile, documentation, and other project materials. It is particularly useful when the application is developed across multiple modules or revised during testing.

GitHub can host the Git repository and provide a convenient way to share the project with faculty members or teammates. However, Git and GitHub do not execute the scheduler automatically; they manage and store the project files and their history.

Before submission, the repository should contain the final source files, build instructions, README, and required documentation. Generated binaries, temporary test data, and private information should be excluded when appropriate.

## 3.3 Operating System Environment – Ubuntu / WSL

The application is designed for a Linux/POSIX environment. Ubuntu running through WSL provides the command-line tools and operating-system interfaces required to compile and execute

the program.
The Linux environment supports the project's use of process-control functions, file operations, pipes, and POSIX threads. It also provides standard command-line utilities such as date, echo, true, and false, which are used as approved programs for demonstrating scheduled execution.
The operating-system environment is important because the scheduler relies on Linux behavior rather than Windows-specific process-management interfaces. For consistent results, the project should be built and run in the same configured Linux environment used during testing.

The source code can be edited using VS Code, while Ubuntu through WSL provides the environment in which the application is compiled and executed.

### 3.3.1 Role of WSL in the Project

WSL provides a Linux user-space environment on a Windows computer. It allows the project to be developed without requiring a separate physical Linux machine.

The developer can edit the project in Visual Studio Code, open the Ubuntu terminal, compile the C source files using GCC, and run the scheduler from the Linux command line. The same terminal can be used to inspect stored job records and execution logs.

WSL is particularly useful for this project because the implementation uses Linux/POSIX interfaces. It provides access to process creation, program execution, waiting for child processes, file handling, pipes, and POSIX thread functions supported by the installed environment.

The scheduler remains an ordinary user-space application. WSL provides the environment in which the application runs; it does not itself perform the application's job scheduling logic

## 3.3.2 Terminal-Based Execution

The project uses a command-line interface instead of a graphical user interface. When the executable starts, the application displays a menu through which the user can schedule a task, view scheduled jobs, inspect job status and logs, cancel a pending task, or exit.

The user selects one of the approved programs: date, echo, true, or false. If echo is selected, a custom message is entered. The user then provides a future date and time in the required format. After the job is accepted, the application records its information and displays the assigned job ID and pending status. When the scheduled time arrives, the running scheduler dispatches the task and records the execution result.

The terminal also displays status information and can be used to inspect the application's log files. This makes the project suitable for a live demonstration because the user can observe the scheduling request and its result directly.

The terminal-based design keeps the implementation focused on scheduling and operating-system functionality instead of graphical interface development.

### 3.3.3 Compilation and Execution Environment

The application is compiled and executed inside Ubuntu/WSL using GCC and GNU Make. The Makefile specifies how the source files are compiled and linked.

The general workflow is:

1. Open the project folder in Visual Studio Code with the Ubuntu/WSL environment selected.
2. Open the integrated Ubuntu terminal.
3. Navigate to the project directory.
4. Check that GCC and GNU Make are available.
5. Compile the application using the project's Makefile.
6. Run the generated executable from the Linux terminal.
7. Schedule a test job and verify its actual execution result.
8. Inspect the job record and associated log.
9.

The commands used to build and launch the program must match the targets defined in the final Makefile. For example, make can be used to build the application when the Makefile provides the default build target.

The application uses the local operating-system clock for scheduling. Its actual execution time can differ slightly from the requested time because of scheduler waiting behavior and operating-system scheduling.

A key limitation is that the application runs in the foreground. The scheduler must remain active to monitor and execute pending jobs. Closing the application stops the scheduling loop; persistent job records alone do not provide background execution.

## 3.4 Development Tools and Compilers

The development tools support the implementation, compilation, testing, and maintenance of the Linux Job Scheduler. The project uses a modular source structure so that different responsibilities can be implemented and examined separately.

Visual Studio Code provides the editing environment, GCC compiles the C source files, GNU Make automates the build process, and Git tracks source-code changes. Ubuntu/WSL supplies the Linux execution environment.

The tools work together to provide a repeatable development process. Source code is edited and reviewed in the editor, compiled through the Makefile, executed in the Linux terminal, and revised when testing identifies errors.

### 3.4.1 Source-Code Editor

Visual Studio Code is used to create, modify, and review the project's C source files, header files, Makefile, and documentation.

The project contains separate modules for menu handling, job management, scheduling, process management, and job storage. The editor's file explorer and search features help the developer

navigate these modules and trace function calls across files.
The integrated terminal allows compilation and execution commands to be run in the same workspace. This helps the developer compare the source code with actual program output and error messages.

The editor can also display Git changes and support extensions for C/C++ development. Such extensions can improve code navigation and error visibility, but they do not replace compilation and testing.

### 3.4.2 Compiler

GCC is responsible for translating the project's C source files into an executable. During compilation, it checks the source code for syntax errors and reports problems that prevent successful compilation.

Because the application is divided into several modules, the build process must compile the required source files and link them together. Header files provide declarations shared between modules, while implementation files contain the corresponding function definitions.

The build configuration must also account for required libraries and compiler options. If POSIX threads are used, the build should include the appropriate pthread support.

A successful compilation confirms that the compiler can build the program. It does not, by itself, prove that scheduling, cancellation, process execution, persistence, and logging behave correctly. Those behaviors require separate runtime tests.

### 3.4.3 Project Compilation Structure

The project uses a Makefile to organize compilation. A Makefile defines build targets and the commands required to produce the executable from the source files.

The main source modules are organized according to their responsibilities:

- main.c — application entry point and startup flow.
- menu.c — terminal menu and user interaction.
- job_manager.c — job creation, status management, and related job operations.
- scheduler.c — checking scheduled times and dispatching due jobs.
- process_manager.c — execution of approved programs and collection of process results.
- job_storage.c — reading and writing persistent job information and associated storage operations.
- Header files — declarations and shared data structures used across modules.
- Makefile — compilation and linking rules.
-

The exact responsibilities should be confirmed against the final source code. The purpose of this structure is to separate application concerns rather than place all logic in one source file.

A normal build is initiated with:
make
The developer should review the build output and resolve any errors or warnings that could affect correctness. If the project provides a test target, that target can be used to execute the defined

automated tests.

### 3.4.4 Development Workflow

The development workflow begins by reviewing the project requirements and organizing the source code into modules. The developer then implements the terminal menu, task validation, job storage, scheduling logic, process execution, and result reporting.

After implementation, the source code is compiled using GCC through the Makefile. Compilation errors are corrected before runtime testing begins.

The application is then tested with the four approved programs. The date program checks whether a task executes and displays its actual execution time. The echo program checks whether a saved custom message is passed and recorded correctly. The true program checks successful execution, while false checks whether a nonzero exit status is correctly reported as failure.

Additional tests should cover invalid dates, past scheduling times, unsupported program choices, cancellation of pending jobs, persistence across restarts, and log inspection. The actual test results should be recorded rather than assumed.

Git is used to track changes during development. Before final submission, the developer should verify the repository contents, confirm that the README instructions work, and ensure that the report and presentation accurately describe the implemented features.

This workflow helps maintain a clear connection between requirements, source-code implementation, compilation, runtime behavior, and the final project documentation.

# CHAPTER - 4 TECHNOLOGY AND TOOLS USED

## 4.1 C Programming

C is the primary programming language used to develop the Linux Job Scheduler and Task Automation System. It is suitable for systems programming because it provides access to operating-system interfaces, process-control functions, file operations, and memory-management facilities.

The application is organized into separate source files for menu handling, job management, scheduling, process execution, and storage. This modular structure helps separate responsibilities and makes the source code easier to understand, maintain, and test.

C is used to collect task details, validate input, maintain job records, calculate when a job becomes due, launch approved programs, and record execution results. The application also uses data structures to represent scheduled jobs and their associated information.

The program supports four approved test programs: date, echo, true, and false. The user can select a program and specify a future execution time. For echo, the user can also provide a custom message. The scheduler records the actual execution result instead of assuming that every task succeeds.

C therefore provides the programming foundation for connecting the scheduling logic with Linux process management and file handling.

## 4.2 GCC Compiler

The GNU Compiler Collection (GCC) is used to compile the C source files and produce the executable program. The compiler translates the source code into machine code and reports errors that must be corrected before the program can be built.

The project contains multiple source files and header files. GCC compiles the implementation files and links the required components into the final executable. The project Makefile defines the build process and any required compiler or linker options.

The application can be built from the Ubuntu/WSL terminal using:
gcc --version
make

The first command displays the installed GCC version. The second invokes the project's Makefile, provided that GNU Make is installed and a default build target is defined.

If POSIX threads are used, the build configuration must include the required thread-support options. The exact compiler command should follow the supplied Makefile rather than being assumed.

Successful compilation confirms that the source code can be built. Runtime testing is still

necessary to verify scheduling, process execution, cancellation, persistence, and logging.

## 4.3 Linux System Calls and APIs

Linux provides system calls and programming interfaces that allow applications to request services from the operating system. These interfaces support process creation, program execution, process termination, file access, communication, and other system operations.

The Linux Job Scheduler uses operating-system interfaces to execute approved programs when their scheduled times arrive. The scheduler itself is a user-space application: it decides when a task is due, while Linux provides the facilities needed to create and manage the executing process.

The following subsections describe relevant interfaces. Each interface should be identified as implemented only if it is present in the final source code.

### 4.3.1 Process Identification APIs

Linux assigns a process identifier (PID) to each process. Process-identification functions can be used to obtain information about the currently executing process or a related process.

The getpid() function returns the PID of the calling process. The getppid() function returns the PID of its parent process.

These functions are useful when examining process relationships during execution and debugging. In a scheduler, process identifiers can help distinguish the main application from a child process created to execute a scheduled program.

The project should describe these functions as implemented APIs only if the source code actually calls them. Process creation and waiting can be demonstrated using fork() and waitpid() even if the program does not explicitly display process IDs.

### 4.3.2 Operating-System Information

Linux exposes system information through various interfaces and command-line utilities. Such information can include the operating-system identity, kernel version, process information, and current system time.

The project can use system utilities such as date to demonstrate output generated at execution time. When a scheduled date job executes, it displays the actual date and time at which the program runs.

System information can also be inspected during development using Linux commands. For example:

```
uname -a
date
```

The uname -a command displays system and kernel information, while date displays the current

date and time.
These commands help document and verify the development environment. They should not be described as functions implemented inside the scheduler unless the application itself invokes or uses them.

### 4.3.3 Process Creation and Control

Process creation and control are central to the project's execution workflow. When a scheduled job becomes due, the process manager launches the selected approved program and obtains its execution result.

The main process-control interfaces relevant to this workflow are:

- fork() — creates a child process.
- An exec() family function — replaces the current process image with the selected program.
- waitpid() — waits for a child process and collects its termination information.

The parent process can inspect the child's termination status using the status information returned through waitpid(). This allows the application to distinguish a successful program exit from a nonzero exit status.

For example, the true program normally returns exit code 0, whereas false returns exit code 1. The scheduler uses the actual result to determine the job status.

The implementation also uses an anonymous pipe to communicate an execution-result record between related processes, according to the supplied implementation description. The pipe and process-control interfaces allow the application to collect results and update the corresponding job record.

The application should use a fixed allowlist of supported programs and should not construct arbitrary shell commands from user input.

### 4.3.4 Alarm and Signal APIs

Signals are software notifications that allow a process to respond to particular events. Examples include SIGINT, commonly associated with a terminal interrupt, and SIGCHLD, associated with changes in a child process's state.

Timer-related facilities and signals can be used in some applications to notify a process when a specified interval has elapsed. However, a timer signal is not the only way to implement a scheduler. An application may instead check the current time and wait until a job becomes due.

The current project should document the exact mechanism used in scheduler.c. If it uses a due-time checking loop and calculates a wait interval, that behavior should be described directly. Do not claim the use of alarm(), setitimer(), SIGALRM, or a signal handler unless the corresponding implementation exists in the source code.

Signals remain a relevant operating-system concept, but their presence in the theory chapter does not establish that they are implemented in the project.

### 4.3.5 File APIs

File input/output is required to preserve job information and execution logs. The scheduler stores records so that jobs can be inspected after they have been created and, subject to the implemented recovery behavior, after the application has been restarted.

C provides standard file functions such as fopen(), fgets(), fprintf(), and fclose(). POSIX also provides file-descriptor interfaces such as open(), read(), write(), and close().

The exact functions used depend on the storage implementation. The job-storage module is responsible for loading and saving the job database and recording or displaying per-job logs. The implementation description indicates that saving uses a temporary database file. This approach can help avoid replacing the main database until the new file has been written, depending on how the save and replacement operations are implemented.

The application should check file-operation return values and report errors when records cannot be read or written. It should not report a successful save when the underlying operation has failed.

### 4.3.6 Thread APIs

POSIX threads provide a way to create and manage threads within a process. They are commonly referred to as pthreads.

The project uses a worker-thread lifecycle for processing due jobs, according to the supplied implementation description. The main thread and worker thread have different execution responsibilities, although they share the process's address space.
Relevant POSIX thread functions include:

- pthread_create() — creates a new thread.
- pthread_join() — waits for a joinable thread to finish.
- pthread_exit() — terminates the calling thread when explicitly used.

The exact functions used should be confirmed from the source code. A worker thread that is joined before the next job is handled demonstrates thread creation and completion, but it does not by itself demonstrate simultaneous execution of multiple jobs.

The project should also distinguish thread management from synchronization. Mutexes, condition variables, and semaphores should be documented as implemented features only if they are actually used.

## 4.4 POSIX Threads / Synchronization

POSIX threads allow a program to divide work into separate execution paths. They are useful when an application needs to separate a particular task from other responsibilities within the same process.

In the Linux Job Scheduler, a worker thread is used for the due-job execution workflow, according

to the supplied project description. The worker's lifecycle is managed by the application so that the execution result can be collected and the job status can be updated.

Synchronization becomes necessary when multiple threads access shared data and their operations could overlap. Without appropriate coordination, a race condition may occur if the final result depends on the order in which the threads access or modify shared data.

The actual synchronization mechanisms must be identified from the final source code. The presence of pthread functions does not automatically mean that the application uses mutexes or condition variables.

### 4.4.1 Monitor Thread

A monitor thread is a thread responsible for observing a condition or checking whether a particular event requires action. In a scheduler, monitoring may involve checking pending jobs and determining whether their scheduled times have arrived.

The project's scheduler module is responsible for checking due jobs. According to the supplied implementation description, a POSIX worker is created and joined for each due job. The exact division of responsibilities between the main thread and scheduler thread should be confirmed from the source code before describing a dedicated monitor thread.

If the application performs scheduling checks in the main thread and creates workers only when jobs become due, it should be documented that way rather than claiming a separate continuously running monitor thread.

### 4.4.2 Mutex

A mutex is a synchronization primitive used to protect shared data from conflicting simultaneous access. A thread locks the mutex before entering a protected section and unlocks it after completing the operation.

Mutexes can be useful in a scheduler if multiple threads may update the same job record or access shared scheduling information concurrently. They can help prevent race conditions when used correctly.

However, a mutex should be listed as an implemented project feature only if the source code initializes, locks, unlocks, and destroys an actual mutex. If the current application executes one worker at a time and does not use a mutex, this section should explain mutexes as a related concept rather than state that the project implements one.

### 4.4.3 Thread Synchronization

Thread synchronization coordinates the actions of multiple threads. POSIX provides mechanisms such as mutexes, condition variables, and semaphores to help threads share resources safely.

In a scheduler, synchronization may be necessary when a thread updates job status while another thread examines pending jobs. Correct coordination helps prevent a job from being dispatched after cancellation or from having its status overwritten by conflicting operations.

The current implementation description states that worker threads are joined before final status

handling continues and that job dispatch is sequential. This provides a way to wait for worker completion, but it should not be presented as evidence of mutex-based synchronization unless mutexes are also present in the code.

The report should identify the synchronization behavior that actually exists. If there is no explicit mutex or condition variable, that limitation should be stated honestly.

### 4.4.4 Thread Completion

Thread completion is the process of ensuring that a thread has finished its assigned work and that its result can be handled safely.

The POSIX function pthread_join() allows one thread to wait for a joinable thread to terminate. After the worker completes, the application can continue with the next stage of its execution workflow.

In the project, the worker-thread lifecycle is used to process due jobs, according to the supplied implementation description. Sequential dispatch means the application waits for one worker to complete before continuing with the corresponding result-handling process.

Correct thread completion management helps prevent unmanaged thread lifecycles and makes it easier to determine when execution results are available. The exact return-value and error-handling behavior should be verified against the final source code.

## 4.5 Linux Commands and Utilities

Linux command-line utilities support the development, compilation, execution, and testing of the project. They allow the developer to navigate the source directory, inspect files, compile the application, run the scheduler, and verify the execution environment.

The commands listed in this section are development and demonstration tools. They should be distinguished from the internal scheduling logic implemented by the C application.

### 4.5.1 wsl

Windows Subsystem for Linux (WSL) provides a Linux environment on a Windows computer. It allows the developer to run Ubuntu and use Linux development tools without requiring a separate physical Linux machine.

The project can be opened in Visual Studio Code and built from the Ubuntu/WSL terminal. The Linux environment provides the POSIX interfaces required by the application.
WSL is the environment in which the program runs; it does not independently schedule the application's jobs.

### 4.5.2 cd

The cd command changes the current working directory in the terminal. It is used to navigate to the project folder before building or executing the application.
For example:
cd /path/to/Linux_Job_Scheduler
The example path is a placeholder. Replace it with the actual project path visible inside Ubuntu/WSL. A Windows path and its corresponding Linux path may differ, depending on where the project is stored.

### 4.5.3 ls

The ls command lists files and directories. It can be used to inspect the project structure and confirm that the source files, headers, Makefile, tests, and other required files are present.
For example:
ls
ls src
ls tests
These commands list the current directory, the source directory, and the test directory, respectively, if those directories exist in the project.
The developer can use the results to confirm that the expected files are present before building the application.

### 4.5.4 gcc

The gcc command invokes the GNU C Compiler. It can display the installed compiler version and, when provided with source files and the appropriate options, compile C code.

For example:
gcc –version

The project's normal build should use the Makefile so that all source files and required compiler options are handled consistently.

### 4.5.5 Running the Scheduler

After the source code has been compiled successfully, the Linux Job Scheduler can be started from the Ubuntu/WSL terminal. The project uses a Makefile to organize the compilation and execution process.

The following command starts the application if the run target is defined in the Makefile:

make run

Alternatively, the generated executable can be launched directly using its actual filename.
When the application starts, the terminal displays the main menu with options to schedule a task, view scheduled jobs, view job status and logs, cancel a pending task, and exit the application.

The user can select one of the approved programs: date, echo, true, or false. For echo, a custom message can be supplied. The user then enters the future date and time at which the program should execute.

After a valid task is scheduled, the application assigns a job ID and records the job with a pending status. When the scheduled time arrives, the running scheduler dispatches the job and records its actual execution result.
The application must remain running to monitor pending jobs. Closing the application stops the foreground scheduler, so pending jobs are not guaranteed to execute while it is closed.

### 4.5.6 Relationship Between Commands

Linux commands support the development, compilation, execution, and verification of the project.

The cd command is used to navigate to the project directory. The ls command helps inspect the source files, header files, Makefile, and test directories. The gcc --version command displays the installed compiler version, while make builds the application using the project's build configuration.

After compilation, make run can launch the application when the corresponding target is available. The user can then schedule test jobs and examine their output, status, and logs through the terminal interface.

These commands form a repeatable development workflow:
**Navigate to the project → Inspect files → Compile the source code → Run the scheduler → Test job execution → Verify results.**

The commands themselves are development utilities; the actual scheduling and job-management behavior is implemented by the C application.

## 4.6 Other Tools / Libraries Used

The project uses supporting tools and libraries to build, execute, test, and maintain the application. The main development environment consists of C, GCC, GNU Make, Ubuntu/WSL, and Visual Studio Code.

POSIX process-control interfaces support the execution and monitoring of scheduled programs. File operations maintain job records and execution logs. POSIX thread functions support the worker-thread lifecycle described in the implementation.

Git may be used to track changes to the source code and maintain the project repository. Automated shell scripts can support repeatable testing, while optional tools such as Valgrind can help identify certain memory-management errors.

These tools support development and verification but do not replace functional testing of the

scheduler.

### 4.6.1 Overall Technology Stack

The technology stack combines programming, compilation, operating-system interfaces, storage, and development tools.

| Technology / Tool | Purpose |
|---|---|
| C programming language | Implements the scheduler and its modules. |
| GCC compiler | Compiles the C source files into an executable. |
| GNU Make | Automates compilation using the Makefile. |
| Ubuntu / WSL | Provides the Linux environment for building and running the application. |
| Visual Studio Code | Supports source-code editing and terminal-based development. |
| Linux process-control APIs | Support process creation, program execution, and waiting for child processes where implemented. |
| POSIX threads | Support the worker-thread lifecycle where implemented. |
| File input/output | Stores job information and execution logs. |
| Anonymous pipes | Communicate execution-result information between related processes where implemented. |
| Git / GitHub | Support source-code version control and repository hosting when used. |
| Shell test scripts | Automate project tests when provided by the repository. |
| Valgrind | Optional tool for memory diagnostics, if installed and used. |

The listed technologies should be checked against the final source code and actual development environment. Features or tools that were not used should not be presented as completed implementation work.

### 4.6.2 Summary

This chapter described the tools, technologies, and operating-system interfaces used in the Linux Job Scheduler and Task Automation System. C is used to implement the application, GCC compiles the source code, and GNU Make organizes the build process. Ubuntu/WSL provides the Linux environment, while Visual Studio Code supports editing and development.

Linux process-control interfaces support the execution of scheduled programs, and file operations maintain job records and execution logs. POSIX threads support the worker-thread lifecycle, while anonymous pipes are used to communicate execution results where implemented.

Linux commands such as cd, ls, gcc, and make support the development workflow. Git, test scripts, and optional diagnostic tools can assist with source-code management and verification.
Together, these technologies provide the foundation for implementing and testing the scheduler. The final evaluation must consider both successful compilation and actual runtime behavior, including scheduling, process execution, status reporting, cancellation, persistence, and logging.

# CHAPTER - 5 SYSTEM ARCHITECTURE AND DESIGN

## 5.1 Overall System Architecture

The Linux Job Scheduler and Task Automation System is developed using the C programming language and is organized into separate modules. Each module is responsible for a specific operation, including user interaction, job management, scheduling, process execution, and file storage.

The main module provides the entry point for the application and starts the terminal menu. The menu module accepts user input and invokes the required operations. The job manager maintains job records and status transitions, while the scheduler checks scheduled times and dispatches jobs when they become due. The process manager executes approved programs and collects their actual execution results. The storage module maintains job records and execution logs.

The overall module structure is shown below.

| Module | Responsibility | Main Functions / APIs |
|---|---|---|
| main.c | Application entry point and startup | `main()` |
| menu.c | Terminal menu and user interaction | `menu_run()`, `schedule_task()`, `show_jobs()`, `show_job_status_and_logs()`, `cancel_pending_task()`, `exit_scheduler()` |
| Job_manager.c | Job creation, validation, status updates, and cancellation | `job_manager_load()`, `job_manager_schedule()`, `job_manager_update()`, `job_manager_cancel()`, `validate_job_record()` |
| scheduler.c | Checks scheduled times and dispatches due jobs | `scheduler_run_due()`, `scheduler_milliseconds_until_next()` |
| process_manger.c | Validates approved programs, executes processes, and collects results | `process_manager_validate()`, `process_manager_run()`, `fork()`, `exec()` family, `waitpid()`, anonymous pipe APIs |
| job_storage.c | Persistent job records and execution logs | `job_storage_load()`, `job_storage_save()`, `job_storage_log()`, `job_storage_show_log()` |

| | | |
|---|---|---|
| POSIX thread support | Worker-thread lifecycle for due-job execution | `pthread_create()`, `pthread_join()` where implemented |
| Linux file and process interfaces | File operations and process-result handling | File I/O APIs and process-control interfaces used by the implementation |

The table describes the modules and APIs according to the project structure and implementation details previously provided. The exact function names and APIs should be checked against the final source files before submission.

The architecture separates responsibilities while allowing the modules to exchange job information and execution results. This makes the application easier to maintain, test, and explain.

The system follows a sequence in which the user schedules a job, the job manager stores the request, the scheduler waits until the job is due, and the process manager executes the selected approved program. The result is then recorded in the job status and associated log.

The application uses four approved programs: date, echo, true, and false. It records the actual execution outcome, allowing successful and failed executions to be distinguished.
The scheduler operates in the foreground and must remain running to monitor pending jobs. The architecture does not include a persistent background daemon.

## 5.2 System Workflow

The system workflow describes the sequence of operations performed by the Linux Job Scheduler and Task Automation System, starting from application startup and continuing through task scheduling, execution, and result recording.

The application follows a modular workflow in which each module performs a specific responsibility. The menu module accepts user input, the job manager maintains job information, the scheduler identifies due jobs, the process manager executes the selected program, and the storage module saves job records and logs.

The workflow consists of the following steps:

**Step 1: Application Initialization**
The application starts through main.c and displays the terminal menu. The job-management and storage components load the available job information according to the implemented startup procedure.

**Step 2: User Operation Selection**
The user selects one of the menu options: Schedule a Task, View Scheduled Jobs, View Job Status and Logs, Cancel a Pending Task, or Exit.

**Step 3: Program Selection and Input**
When scheduling a task, the user selects one of the approved programs: date, echo, true, or false. If echo is selected, the user also provides a custom message. The user then enters the scheduled date and time.

**Step 4: Input Validation**
The application validates the program selection and date/time input. Invalid formats, invalid calendar dates, and unsuitable scheduling times are rejected before the job is accepted.

**Step 5: Job Creation and Storage**
The job manager creates a job record, assigns a unique job ID, and sets the initial status to PENDING. The storage module saves the record for subsequent scheduling and inspection.

**Step 6: Scheduled-Time Monitoring**
The scheduler checks pending jobs to determine whether their scheduled execution times have arrived. Jobs that are not yet due remain pending while the application continues running.

**Step 7: Process Execution**
When a job becomes due, the scheduler dispatches it to the process manager. The process manager validates the selected program and launches it using the implemented Linux process-control interfaces.

**Step 8: Execution Result Collection**
The process manager waits for the program to finish and collects its actual exit result. Program output and execution information are captured for logging. The implementation uses an anonymous pipe to communicate execution-result information between related processes.

**Step 9: Status Update and Logging**
The job manager updates the job according to the execution result. A successful execution is recorded as succeeded, while a nonzero program exit status is recorded as failed. The output and result are saved in the associated log.

**Step 10: Return to Menu**
After the operation, the application returns to the terminal menu. The user can inspect job details, view logs, schedule another task, or cancel a pending task.

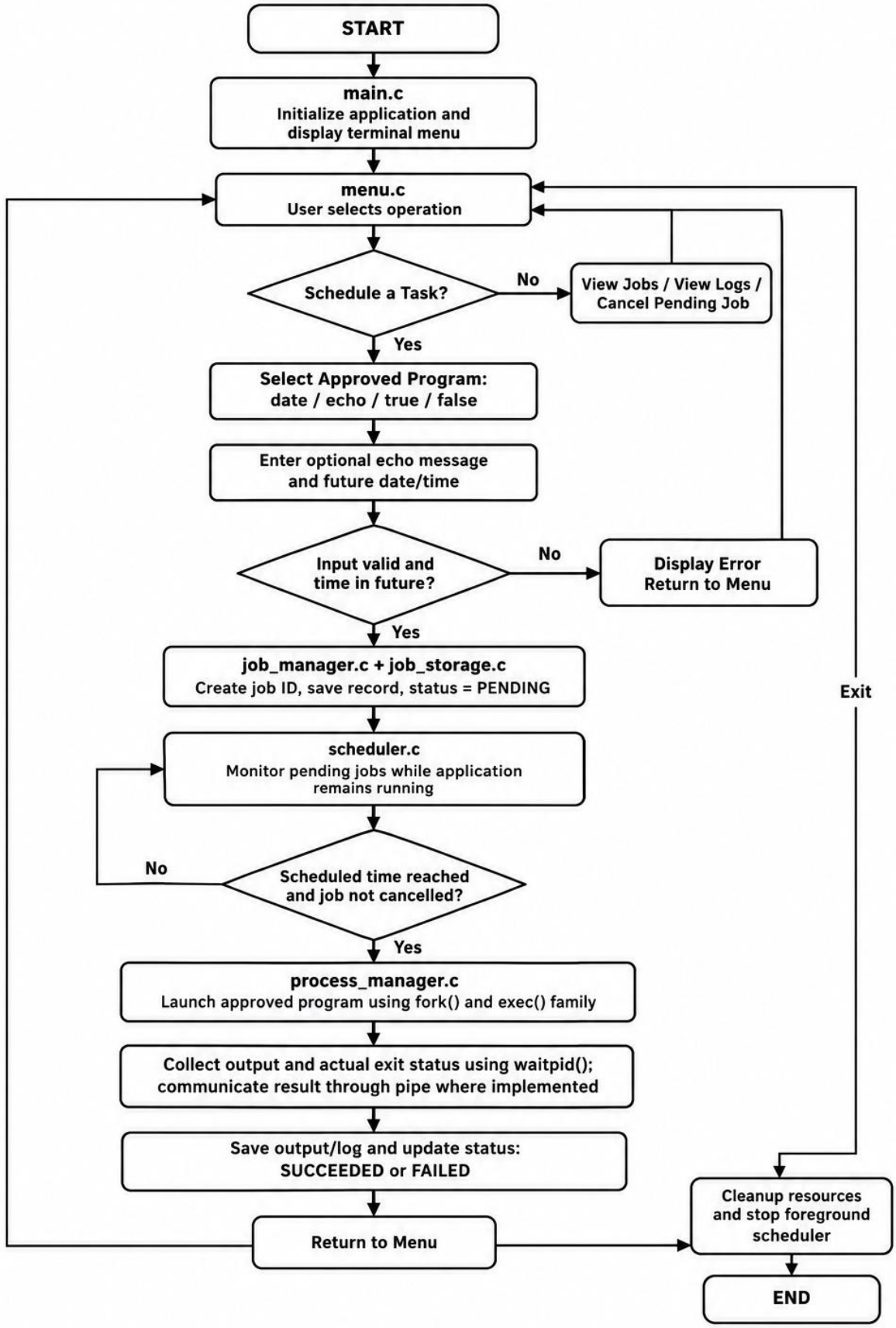
**Step 11: Application Exit**
When the user selects Exit, the application performs the applicable cleanup and stops the foreground scheduler. Pending jobs are not guaranteed to execute while the application is closed. The workflow connects user input, scheduling, process execution, and result recording into one sequence. It ensures that a scheduling confirmation is distinct from proof that the scheduled program has actually executed.

## 5.3 Flowchart / Data Flow

The flowchart represents the control flow of the application, while the data-flow description explains how job information moves between modules.

# LINUX JOB SCHEDULER AND TASK AUTOMATION SYSTEM

## SYSTEM FLOW




Foreground scheduler: pending jobs do not execute while the application is closed.

**Figure 5.1: System flow from menu selection**

## 5.4 Module Description

The system is divided into modules to make its design easier to understand, maintain, and test. Each module performs a specific responsibility and communicates with other modules through the functions and data structures defined in the source code.

### 5.4.1 Main Module — main.c

The main module provides the entry point of the application. Its main() function starts the application and invokes the necessary initialization and menu operations.
It separates application startup from the detailed logic used to manage jobs, check scheduling times, execute programs, and store results.

### 5.4.2 Menu Module — menu.c

The menu module manages interaction with the user through the terminal. It displays the available menu options and processes the user's selections.
Its responsibilities include scheduling tasks, displaying scheduled jobs, viewing job status and logs, cancelling pending tasks, and exiting the application. It also collects the program selection and scheduling information required to create a job.
The menu module passes requests to the job-management functions instead of directly implementing all scheduling and process-execution logic.

### 5.4.3 Job Manager — job_manager.c

The job manager handles the creation and maintenance of job records. It assigns job IDs, validates job information, updates job status, and processes cancellation requests.
When a valid task is submitted, the job manager creates a record with the selected program, scheduled time, and required arguments. It initializes the job status as pending and coordinates with the storage module to preserve the record.
The job manager also helps maintain valid status transitions so that cancellation and execution outcomes are recorded consistently.

### 5.4.4 Scheduler Module — scheduler.c

The scheduler module monitors pending jobs and determines when they become due. It compares the scheduled time of a job with the current time and dispatches jobs that are ready for execution.
The module is responsible for scheduling decisions, including determining the next waiting interval according to the implemented logic. It passes due jobs to the process manager for execution.
The scheduler runs in the foreground. It does not continue monitoring jobs after the application has exited.

### 5.4.5 Process Manager — process_manager.c

The process manager is responsible for executing the selected approved program. It validates the program against the allowlist and uses the implemented Linux process-control interfaces to launch it.
The implementation description identifies fork(), an exec() family function, and waitpid() as part of the process-execution workflow. The process manager collects the actual exit result and uses an

anonymous pipe to communicate execution-result information between related processes.
The program's output is captured for logging. This allows the application to distinguish successful execution from a nonzero exit status and to record execution failures accurately.

### 5.4.6 Job Storage Module — job_storage.c

The job storage module handles persistent job records and execution logs. It loads saved information, writes new or updated job records, and manages the log information associated with individual jobs.
The storage module allows the application to display previously saved job information and inspect execution results. The implementation description indicates that a temporary database file is used during saves.
File operations should be checked for errors so that failed reads or writes are not reported as successful operations.

### 5.4.7 Interaction Between Modules

The modules cooperate throughout the job lifecycle. The menu collects the user's request and passes it to the job manager. The job manager validates the request and stores the job. The scheduler checks when the job becomes due and passes it to the process manager. The process manager executes the program and collects the result, after which the job manager and storage module record the outcome.
This separation of responsibilities reduces unnecessary duplication and makes it easier to test individual parts of the application.
The module-based design also supports future improvements, such as a background service, more advanced scheduling policies, or additional synchronization mechanisms. Such improvements should be treated as future work unless they are already implemented.

## 5.5 Input and Output Design

Input and output design describes how users interact with the Linux Job Scheduler and Task Automation System and how the application presents scheduling information and execution results. The system uses a terminal-based interface developed in C. Users enter task details through the menu, and the application displays job information, status updates, and execution results through the terminal and log files.
The input and output design aims to provide clear interaction, validate scheduling requests, and make the execution status of each job easy to understand.

### 5.5.1 Input Design

The application accepts user input through a terminal menu. The main menu provides the following operations:

1. Schedule a Task
2. View Scheduled Jobs
3. View Job Status and Logs
4. Cancel a Pending Task
5. Exit

When the user selects **Schedule a Task**, the application displays the approved programs:
date — displays the current date and time when executed.

echo — displays a custom message supplied by the user.
true — returns exit code 0 to test successful execution.
false — returns exit code 1 to test failed execution.
For the echo program, the user enters a message that will be passed to the program when the job executes. The message is stored with the job so that it can be used at the scheduled time.
The user then enters the scheduled date and time in the format:
YYYY-MM-DD HH:MM:SS
The application validates the input before accepting the scheduling request. Invalid formats, invalid calendar dates, past scheduling times, and unsupported program selections should be rejected with an appropriate error message.
After successful validation, the application creates a job record, assigns a unique job ID, stores the scheduled information, and sets the initial job status to PENDING.

### 5.5.2 Output Design

The application presents information through terminal messages and job log files. Output is designed to help users understand whether a task has been accepted, whether it is still pending, and what happened when it was executed.
After a task is scheduled successfully, the application displays information such as:

- Job ID
- Selected program
- Scheduled date and time
- Current job status

When a job becomes due, the process manager launches the selected program and collects the actual execution result. The application records the result and updates the job status.
For example, a date task displays the date and time when it actually executes. An echo task displays the custom message entered by the user. The true program normally returns exit code 0, whereas false normally returns exit code 1. These programs help demonstrate both successful and failed execution.
The output should reflect the actual execution result rather than displaying a success message for every task.

### 5.5.3 Job Status Display

The job-status display allows users to inspect the current or final state of a scheduled task. A newly created job is assigned the status PENDING. When execution completes, the status is updated according to the actual result.
The main statuses are:

| Status | Meaning |
|---|---|
| PENDING | The job is waiting for its scheduled execution time. |
| SUCCEEDED | The program completed with a successful exit result. |
| FAILED | The program failed, returned a nonzero exit status, or encountered an execution error, according to the implemented status rules. |
| CANCELLED | The pending job was cancelled and should not be dispatched. |

The user can select the job-status and logs option to inspect a job's execution result. The application should display the exit result when available and distinguish a scheduling confirmation from actual task completion.

### 5.5.4 Log File Design

Log files provide a record of the output and execution information associated with a job. They allow users to inspect what happened after a task was executed.
According to the project design, individual job logs are stored using filenames such as: logs/job_<id>.log

Here, <id> represents the unique job ID assigned to the task. The exact log directory and filename format should match the final implementation.

The log may contain the program's output, execution result, exit code, and other information recorded by the application. For example, the log for an echo job can contain the custom message, while a false job can record its nonzero exit result.

The storage module is responsible for recording and displaying log information. File-operation errors should be handled explicitly so that the application does not report successful logging when a write has failed.

### 5.5.5 Cancellation and Error Messages

The application provides a cancellation option for pending jobs. When the user selects a job ID for cancellation, the application checks whether the job exists and whether its current status permits cancellation.

A successful cancellation changes the job status to CANCELLED. The scheduler must check the job's status before dispatching it so that a cancelled pending task is not executed.

Error messages provide feedback when an operation cannot be completed. Examples include invalid date/time input, unsupported program selection, an unknown job ID, an attempt to cancel a job that is no longer pending, and failures while reading or writing job records or logs.
Clear messages help users correct input and understand the state of the application.

### 5.5.6 Input and Output Summary

The input and output design connects user interaction with the scheduling and execution components. The user supplies a program, optional arguments, and a future execution time. The application validates the request, creates a job record, and monitors the job while the foreground scheduler remains active.

When execution occurs, the system records the program output and actual execution result. Job-status information and log files allow users to inspect the outcome later.

This design provides a straightforward terminal-based interface for demonstrating scheduled execution, process control, job persistence, output capture, and error handling. The scheduler does not continue monitoring pending jobs after the application has exited.

# CHAPTER - 6 IMPLEMENTATION

## 6.1 Module-wise Implementation

### 6.1.1 main.c

The main.c file serves as the entry point of the Linux Job Scheduler and Task Automation System. It initializes the application and starts the terminal-based menu. It coordinates the main program flow and allows users to access the available scheduling operations.

### 6.1.2 menu.c

The menu.c file provides the terminal menu through which users interact with the application. It allows users to schedule tasks, view scheduled jobs, check job status and execution logs, cancel pending jobs, and exit the application. It collects user input and directs the selected operation to the appropriate module.

### 6.1.3 job_manager.c

The job_manager.c file manages job creation and job-status updates. It handles job identification, validates scheduling requests, and maintains job states such as PENDING, SUCCEEDED, FAILED, and CANCELLED. It also supports cancellation of jobs that have not yet been executed.

### 6.1.4 scheduler.c

The scheduler.c file checks pending jobs and compares their scheduled execution times with the current time. When a job becomes due, the scheduler dispatches it for execution through the process-management component. The scheduler continues checking jobs while the application is running in the foreground.

### 6.1.5 process_manager.c

The process_manager.c file handles the execution of approved programs. It restricts execution to supported programs, starts the selected program using the implemented process-control mechanisms, and collects its execution result. The result is used to determine whether the job completed successfully or failed, and the relevant output is recorded for later inspection.

### 6.1.6 job_storage.c

The job_storage.c file manages the storage of job information and execution logs. It supports saving job records and loading previously stored records when the application starts. It also helps maintain execution information so users can review job status and results after execution.

## 6.2 Algorithm / Pseudocode

The scheduling algorithm describes how a task is accepted, validated, stored, executed at the scheduled time, and recorded after completion. The algorithm separates task scheduling from

process execution so that each component has a clear responsibility.

The application supports four approved programs: date, echo, true, and false. The user selects a program and provides a future date and time. For echo, the user also enters a custom message.

### 6.2.1 Algorithm for Scheduling and Executing a Job

**Input:** Selected program, optional program arguments, and scheduled date/time.
**Output:** Job record, execution result, updated status, and log information.

1. Start the application and initialize the required modules.
2. Load previously stored job records.
3. Display the terminal menu.
4. Ask the user to select an operation.
5. If the user selects Schedule a Task:
    - Display the approved programs.
    - Read the selected program.
    - If the program is echo, read the custom message.
    - Read the scheduled date and time.
6. Validate the program selection, date/time format, calendar values, and future time.
7. If validation fails, display an error and return to the menu.
8. If validation succeeds:
    - Assign a unique job ID.
    - Create a job record.
    - Set the job status to PENDING.
    - Save the job record.
9. While the scheduler is running:
    - Check pending jobs.
    - Determine whether a job's scheduled time has arrived.
    - If the job is not due, continue waiting or checking.
    - If the job is cancelled, do not execute it.
10. When a job becomes due, pass it to the process manager.
11. Validate the selected program against the allowlist.
12. Launch the approved program using the implemented process-control interfaces.
13. Wait for execution to complete and collect the actual exit result.
14. Capture the program output and record the execution information.
15. Update the job status:

- SUCCEEDED for a successful execution.
- FAILED for a failed execution.

16. Save the updated record and associated log information.
17. Return to the menu and continue until the user selects Exit.
18. Clean up applicable resources and stop the foreground scheduler.

### 6.2.2 Pseudocode

```
BEGIN

  initialize_application()
  load_saved_jobs()

  WHILE application_is_running
```

```
display_menu()
choice = read_user_choice()

IF choice == SCHEDULE_TASK

   program = select_approved_program()

   IF program == echo
      message = read_custom_message()
   END IF

   scheduled_time = read_scheduled_time()

   IF input_is_invalid(program, scheduled_time)
      display_error()
      CONTINUE
   END IF

   job = create_job(program, scheduled_time, message)
   job.id = assign_unique_job_id()
   job.status = PENDING

   save_job(job)

ELSE IF choice == VIEW_JOBS

   display_saved_jobs()

ELSE IF choice == VIEW_STATUS_AND_LOGS

   display_selected_job_status_and_logs()

ELSE IF choice == CANCEL_PENDING_JOB

   cancel_selected_pending_job()

ELSE IF choice == EXIT

   stop_application()
   BREAK

END IF

check_pending_jobs()

FOR each pending_job

   IF pending_job_is_cancelled
      CONTINUE
   END IF
```

```
        IF scheduled_time_has_arrived

            result = execute_approved_program(pending_job)

            record_output_and_exit_result(result)

            IF result_is_successful
               update_status(SUCCEEDED)
            ELSE
               update_status(FAILED)
            END IF

            save_job_and_log()

        END IF

      END FOR

   END WHILE

   cleanup_resources()

END
```

The pseudocode illustrates the overall logic rather than the exact source-code structure. Actual scheduling checks, error handling, thread creation, and storage operations must follow the implementation in the project files. The scheduler operates in the foreground and does not continue executing pending jobs while the application is closed.

## 6.3 Core C Implementation

The project is implemented in C and divided into modules to separate user interaction, job management, scheduling, process execution, and persistent storage. This modular approach makes the code easier to maintain and allows each component to be examined independently.

### 6.3.1 Data Structures

The job data structure represents a scheduled task. Depending on the final implementation, it contains information such as:

- Unique job ID.
- Scheduled execution time.
- Selected approved program.
- Optional program arguments, such as an echo message.
- Current job status.
- Execution result or exit code, when available.

The job manager uses this information to create, validate, update, and cancel jobs. The storage

module saves the required fields to persistent storage so that job information can be loaded again. The exact field names and data types should match the structure declared in the project's header files. The project should not be described as using a particular structure layout unless that layout exists in the source code.

### 6.3.2 Input Validation

Input validation prevents invalid scheduling requests from being accepted. The application checks the selected program and verifies the supplied date and time before creating a job record.

Validation should reject unsupported program names, malformed date/time input, invalid calendar dates, and times that are not in the future. Input-reading functions should also enforce appropriate length limits so that long input cannot overflow fixed-size buffers.

The echo message is treated as an argument to the approved program rather than being concatenated into an unrestricted shell command. This design helps reduce command-injection risks.

### 6.3.3 Job Management and Status Handling

The job manager assigns a job ID and initializes a newly created job with PENDING status. The scheduler uses that status to identify jobs that may need execution.

When a user requests cancellation, the job manager checks that the job exists and is still pending. The scheduler must check the job state before dispatching it so that a cancelled job is not launched.

After execution, the actual process result is used to determine the final status. The true program provides a successful exit test, while false provides a nonzero-exit test. This allows the status-handling logic to be tested with both outcomes.

### 6.3.4 Error Handling

The implementation should handle errors during input validation, storage operations, process creation, program execution, waiting for child processes, and communication through pipes.

If a file cannot be opened or written, the application should report the error instead of claiming that the data was successfully saved. If process creation or execution fails, the application should distinguish that failure from a program that ran and returned a nonzero exit status.

The job manager and storage module should also validate persisted records when loading them. Malformed or inconsistent records should not silently be accepted as valid jobs.

Error handling improves reliability and helps the user understand why an operation failed.

## 6.4 Linux System Calls / APIs Used

Linux and POSIX provide interfaces that allow applications to create processes, execute programs, communicate between processes, access files, and manage threads. The project uses these interfaces to implement the scheduling and execution workflow.

The following APIs are relevant to the implementation described for this project. Their exact use should be confirmed in the source code.

### 6.4.1 fork()

The fork() system call creates a child process. The child initially receives a copy of the calling process's execution context, after which the parent and child can follow different execution paths.

In the scheduler, process creation allows the application to launch a separate process for an approved program. The child process can then execute the selected program.

### 6.4.2 exec() Family

The exec() family of functions replaces the current process image with another program. It is commonly used in the child process after fork().
The process manager uses an exec() family function to launch the approved program. The selected executable and its arguments should be passed safely without constructing arbitrary shell commands from user input.
The exact function, such as execv() or execvp(), must match the function actually used in the source code.

### 6.4.3 waitpid()

The waitpid() system call allows a parent process to wait for a specified child process and collect its termination status.
The process manager uses the child-process result to determine whether execution succeeded or failed. A successful exit normally has exit code 0, while a nonzero exit code indicates that the program reported failure.
The application must distinguish a normal process exit from termination by a signal and handle errors returned by waitpid().

### 6.4.4 Pipe APIs

An anonymous pipe provides a communication channel between related processes. The pipe() function creates the pipe, while read() and write() can be used to transfer data through its file descriptors.
The implementation description identifies an anonymous pipe for communicating execution-result information between related processes. The parent process can read the result and use it to update the corresponding job record.
The pipe's file descriptors must be closed when they are no longer needed. Reads, writes, and communication failures should be handled appropriately.

### 6.4.5 File APIs

File operations support persistent job storage and execution logging. Depending on the implementation, the project may use standard C file functions or POSIX file-descriptor functions. Examples include fopen(), fprintf(), fgets(), and fclose() for standard C file handling, or open(), read(), write(), and close() for POSIX file descriptors.
Only the functions actually used in job_storage.c should be listed as implemented APIs. Their purpose is to load and save job information and maintain the associated execution logs.

### 6.4.6 POSIX Thread APIs

POSIX thread functions manage the worker-thread lifecycle used in the scheduling workflow.
The implementation description identifies worker-thread creation and joining for due-job execution. The relevant functions are pthread_create() and pthread_join() if those calls are present in the source code.
Thread creation does not automatically imply concurrent execution of multiple jobs. According to the previously supplied implementation description, jobs are dispatched sequentially, with the

worker joined before processing continues.

## 6.5 Process / Thread Implementation

Process and thread management are important parts of the scheduler's execution workflow. The scheduler determines when a job is due, while the process manager launches the selected program and collects the actual result.

### 6.5.1 Process Creation

When a job reaches its scheduled time, the scheduler passes the job information to the process manager. The process manager validates the selected program against the approved allowlist before starting execution.

The described implementation uses fork() to create a child process and an exec() family function to launch the selected program. The parent process waits for the child using waitpid() and collects the termination result.

This arrangement separates the scheduling application from the execution of the selected program. It also allows the application to capture the actual process outcome rather than assuming that execution always succeeds.

### 6.5.2 Exit-Status Collection

After the child process finishes, the parent examines the termination information returned through waitpid(). It distinguishes a successful exit from a nonzero exit status and other process-termination conditions.

For the four approved test programs:

date displays the actual date and time when it executes.

echo displays the custom message supplied by the user.

true normally returns exit code 0.

false normally returns exit code 1.

The process result is used to update the job status and record the execution information in the log.

### 6.5.3 Worker-Thread Lifecycle

The implementation description indicates that a POSIX worker thread is created for each due job and joined before processing continues. The worker participates in the job-execution workflow, while the scheduler remains responsible for identifying due jobs.

This lifecycle provides a clear point at which the application can wait for the worker to finish and continue with result handling. Because the worker is joined before processing continues, this design should be described as sequential dispatch rather than parallel execution of multiple jobs.

The report should identify the actual thread functions and responsibilities from the source code.

It should not claim the use of a dedicated monitor thread, mutexes, or condition variables unless those mechanisms are implemented.

## 6.6 IPC / Synchronization Implementation

Inter-process communication (IPC) allows processes to exchange information. Synchronization coordinates operations so that processes or threads do not interfere with each other.

The project description identifies an anonymous pipe for communicating execution-result information between related processes. Thread creation and joining are also used in the described worker-thread lifecycle.

### 6.6.1 Anonymous Pipe Communication

An anonymous pipe is created using the pipe() system call and provides a pair of file descriptors for communication. One end can be used for writing, while the other is used for reading.
In the process-execution workflow, the pipe carries an execution-result record between related processes. The parent process reads the result and uses it to update the job's status and execution information.
The communication protocol must be consistent between the writer and reader. Both sides must handle errors, and unused file descriptors should be closed to prevent resource leaks or unexpected blocking.
The exact record format and the location of pipe creation should be documented from the actual implementation.

### 6.6.2 Thread Completion and Coordination

The worker-thread lifecycle uses thread completion management so that execution results can be handled after the worker finishes. If pthread_join() is used, the calling thread waits for the joinable worker to terminate.
This mechanism coordinates thread completion but should not be confused with mutex-based protection of shared data. A mutex is required only when the implementation needs to protect shared state from concurrent access.
The report should state whether the project actually uses mutexes, condition variables, or semaphores. If these mechanisms are absent, they should be discussed as related concepts rather than claimed as implemented features.

### 6.6.3 Cancellation and Dispatch

Cancellation requires the scheduler to avoid dispatching a job that has already been cancelled. The job manager changes the status of an eligible pending job, and the scheduler checks that state before launching it.
If cancellation and dispatch can occur concurrently, the implementation must ensure that the state transition and dispatch decision cannot conflict. The actual protection mechanism should be described according to the source code.
This behavior is important because cancellation should affect execution, not merely change the text displayed to the user.

## 6.7 Memory Management Implementation

Memory management is necessary to ensure that the application uses allocated memory and operating-system resources correctly. In C, memory may be managed using automatic storage, statically allocated objects, or dynamic allocation.
The scheduler stores and processes job information. Depending on the implementation, this information may be held in fixed-size arrays, structures, buffers, or dynamically allocated objects. Input buffers and temporary data structures must be handled carefully to prevent invalid memory access and buffer overflows.

### 6.7.1 Input and Buffer Management

Input functions should enforce buffer limits and ensure that the supplied text fits within the destination buffer. This is important for date/time strings, program arguments, and custom echo messages.
If the implementation uses fixed-size buffers, the code should reject or safely handle input that exceeds the available capacity. It should not write beyond the end of a buffer.

### 6.7.2 Resource Cleanup

The application must manage operating-system resources as well as memory. Child processes must be waited for, pipe file descriptors must be closed, files must be closed after use, and worker threads must be managed according to their lifecycle.
If dynamically allocated memory is used, it should be released when it is no longer required. Error-handling paths must also release resources acquired before the error occurred.
Careful cleanup reduces the likelihood of resource leaks and improves reliability during repeated scheduling and execution.

### 6.7.3 Separate Memory-Management Demonstration

The project description identifies examples/memory_management_demo.c as a separate demonstration program. If that file demonstrates dynamic allocation and deallocation, it should be described separately from the main scheduler.

The existence of a memory-management demonstration does not establish that the scheduler itself uses a dynamically allocated job store. The report should distinguish the demonstration program from the actual job-storage implementation.

Similarly, ordinary use of C variables and structures does not by itself demonstrate virtual-memory management, paging, or address translation. Those concepts should be claimed only when supported by an actual implementation or experiment.

### 6.7.4 Summary

The memory-management approach focuses on safe input handling, correct use of buffers, and cleanup of resources used during scheduling and process execution. The implementation should be checked for correct allocation and deallocation wherever dynamic memory is used.
Together, these practices support a more reliable application and complement the process, thread, IPC, and file-management mechanisms described in this chapter.

# CHAPTER - 7 EXECUTION AND DEMONSTRATION

## 7.1 Compilation in Ubuntu

The Linux Job Scheduler and Task Automation System is implemented in C and organized into separate source and header files. The supplied project package includes a Makefile to build the application. The application executable in the package is named linux-job-scheduler.
From the project root directory in the Ubuntu terminal, the build can be started with the following commands:

makeclean
make

The first command removes generated build outputs where supported by the Makefile. The second command compiles and links the source files. The actual terminal screenshot should be captured during a fresh build on the machine used for the demonstration

## 7.2 Running the Application

After a successful build, launch the application using the run target if available, or execute the generated binary from the project directory. The supplied project uses the executable name linux-job-scheduler.

./build/linux-job-scheduler

The scheduler checks jobs while the application is running in the foreground. It should not be described as a background daemon unless a separate daemon or service has actually been implemented.

## 7.3 Sample Inputs and Outputs

The following screenshot shows an example of scheduling an echo task and the resulting pending job. It provides evidence of the user entering a supported program and setting its scheduled time.

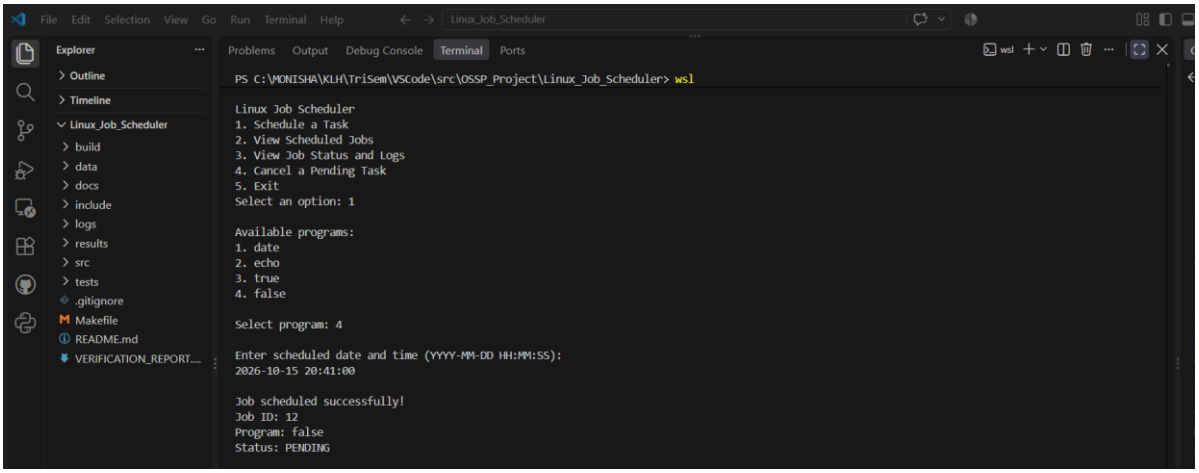



*Figure 7.1: Scheduling an echo job and receiving a pending status.*

The next screenshot shows the job in the scheduled-jobs list. This demonstrates that a scheduled task can be reviewed before execution.

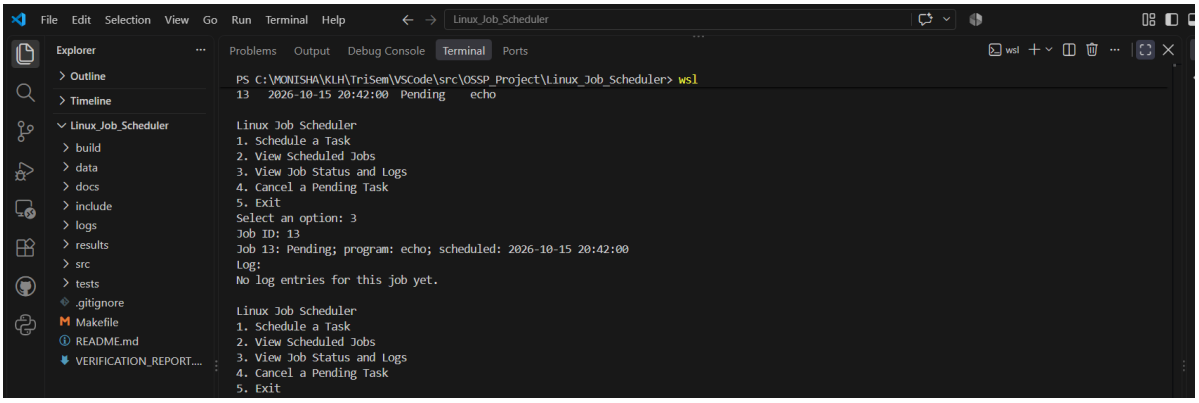



*Figure 7.2: Scheduled echo job displayed in the job list.*

## 7.4 Task Execution and Logs

The project captures task results and makes execution information available through its status or log functionality. The supplied evidence includes an echo job completing with exit code 0.

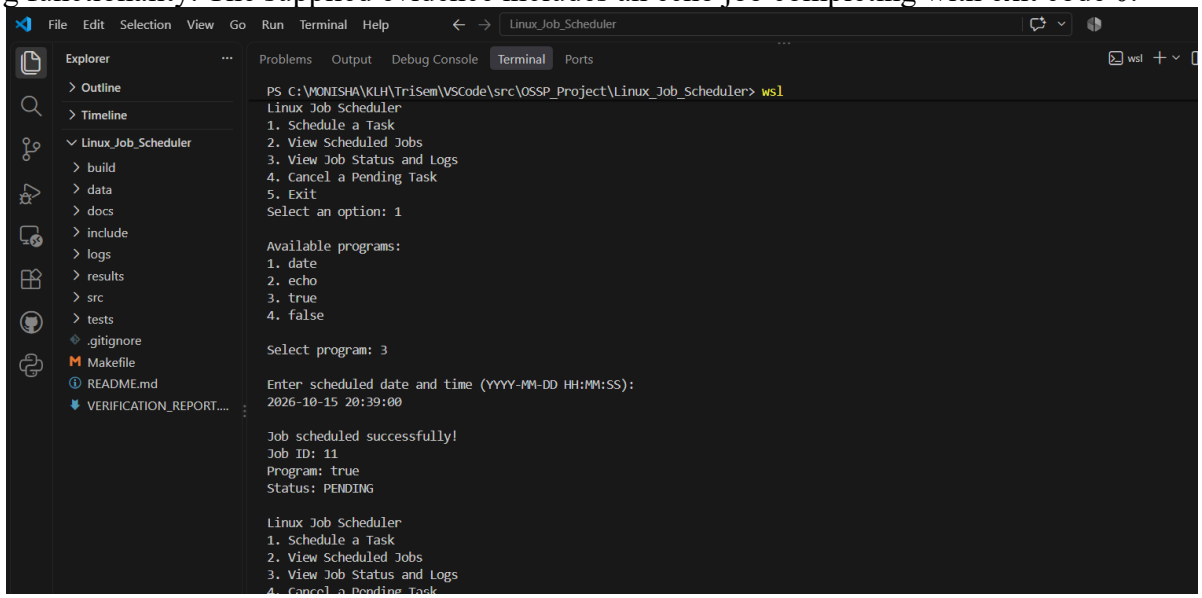



*Figure 7.3: Echo job execution result with exit code 0.*

A completion log provides a record of the executed task. The log view can be used to review the task's outcome after execution.

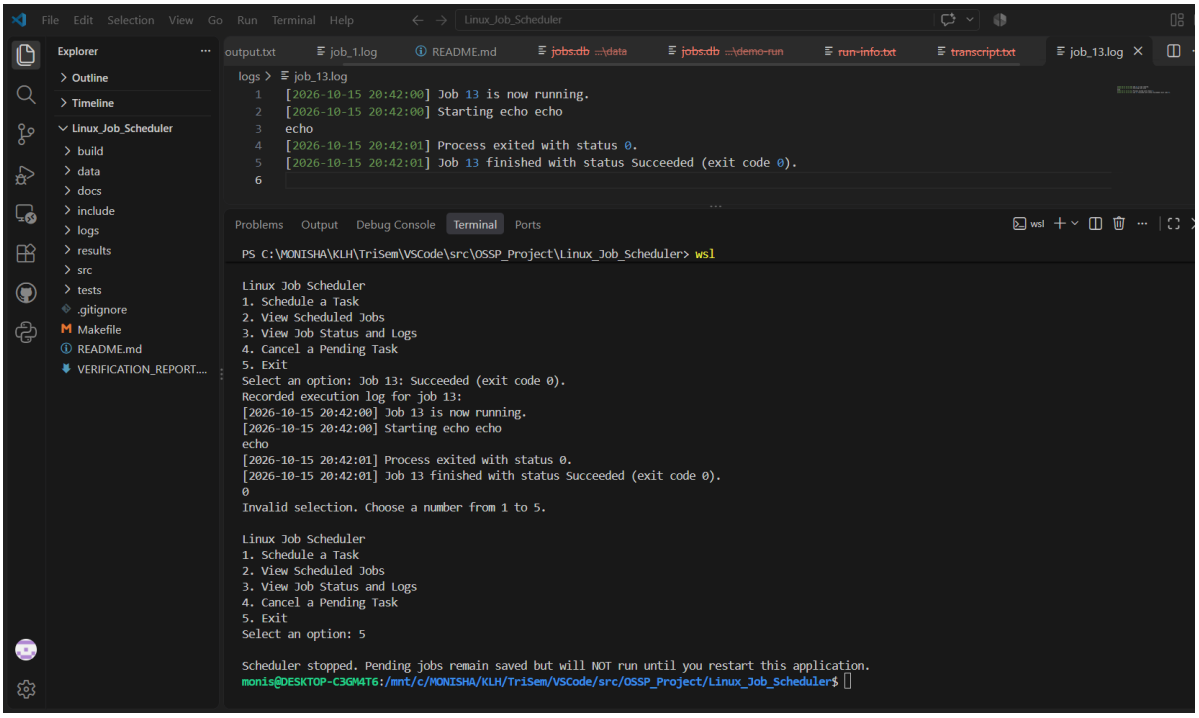



*Figure 7.4: Completion log for the echo job.*

The package also contains evidence for a date task and its execution log.

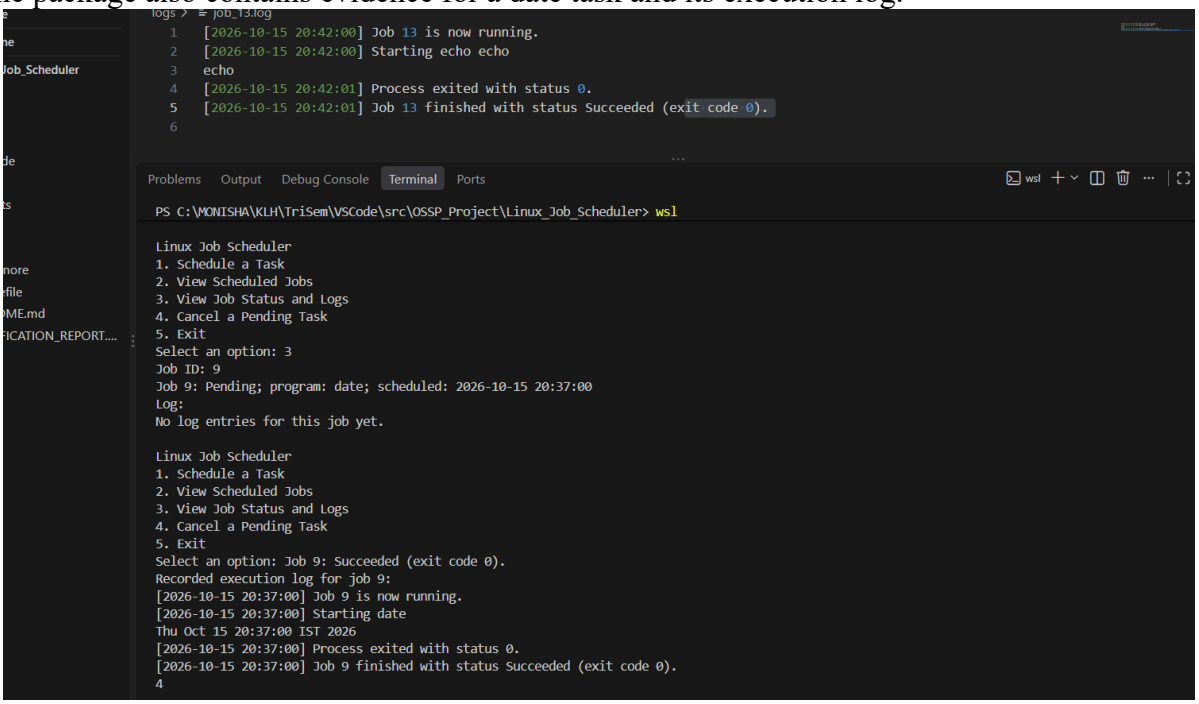



*Figure 7.5: Date job execution log.*

## 7.5 Input Validation and Error Handling

Input validation is important because the scheduler should accept only supported programs and valid future execution times. The supplied screenshots include examples of rejecting an unsupported program and rejecting a scheduled time in the past.

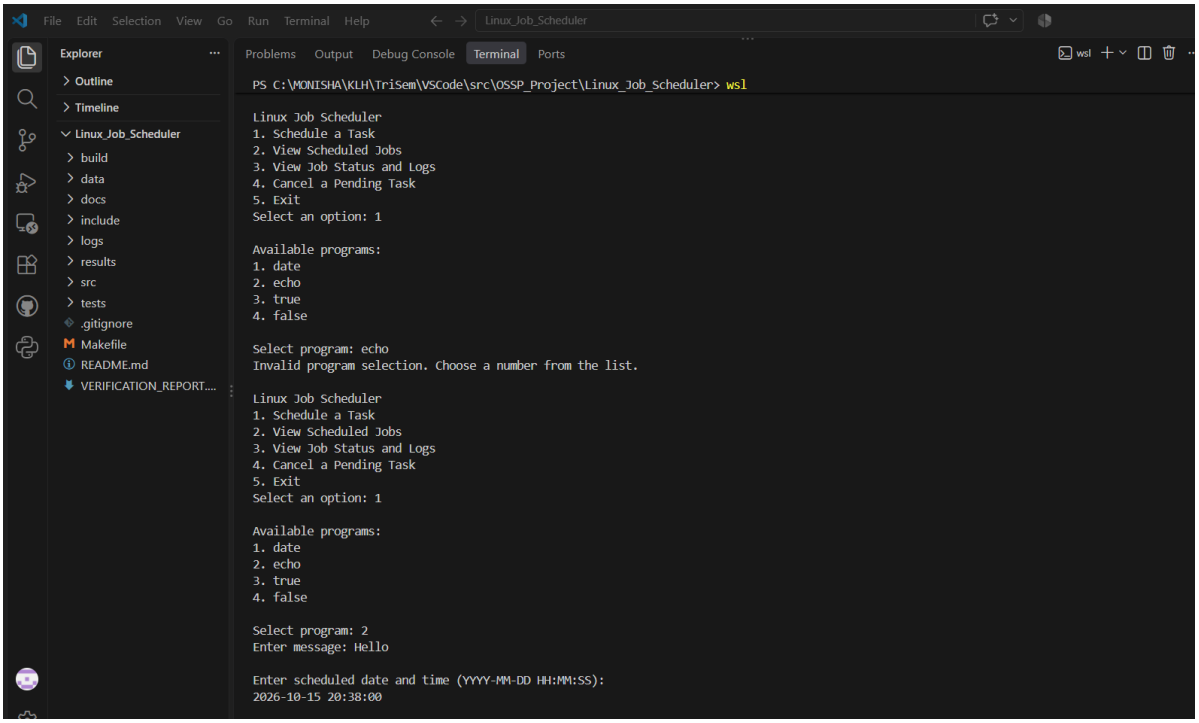



*Figure 7.6: Unsupported program rejected before a supported echo task is scheduled.*

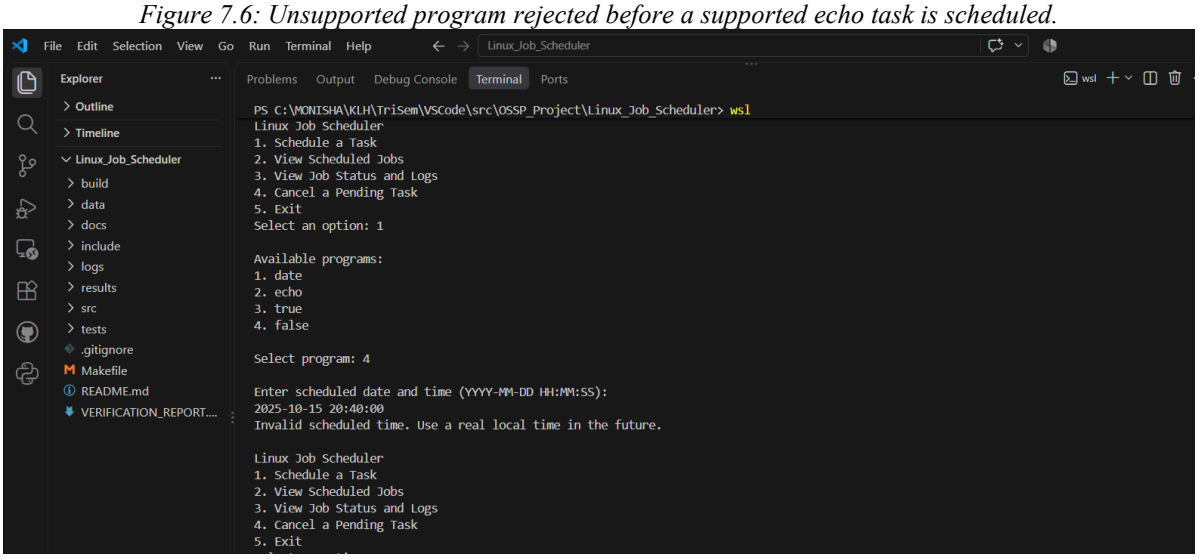



*Figure 7.7: Past scheduled time rejected by input validation.*

## 7.6 Cancellation of Pending Jobs

The application supports cancellation of pending tasks. The supplied screenshots show a pending echo job being cancelled and its status being verified.

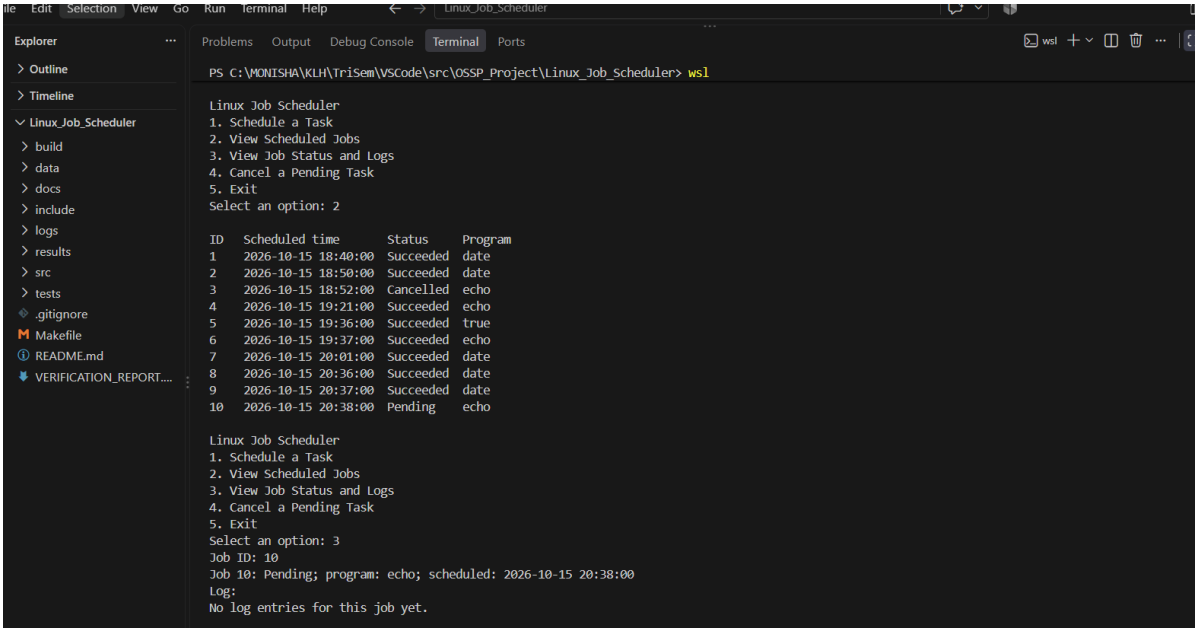



*Figure 7.8: Cancelling a pending echo job.*

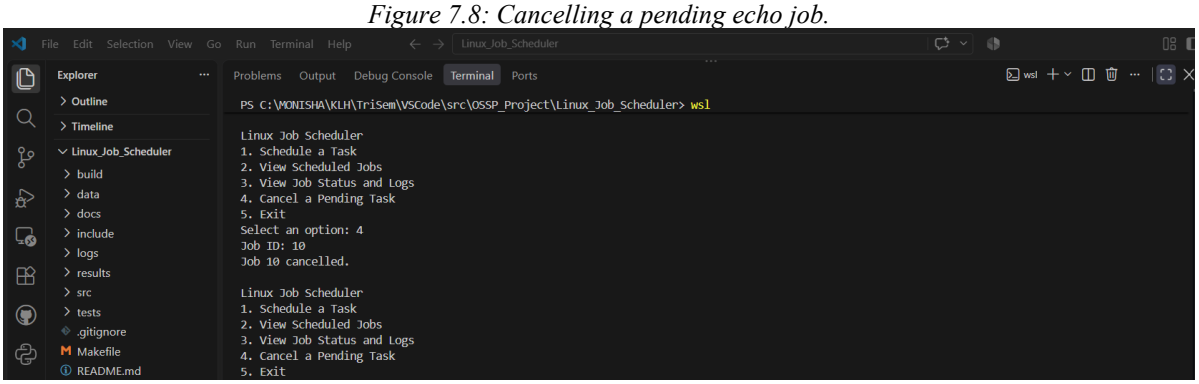



*Figure 7.9: Verifying the cancelled status of the job.*

Cancellation is intended for jobs that have not yet executed. The project package also contains evidence for rejecting an attempt to cancel a completed job.

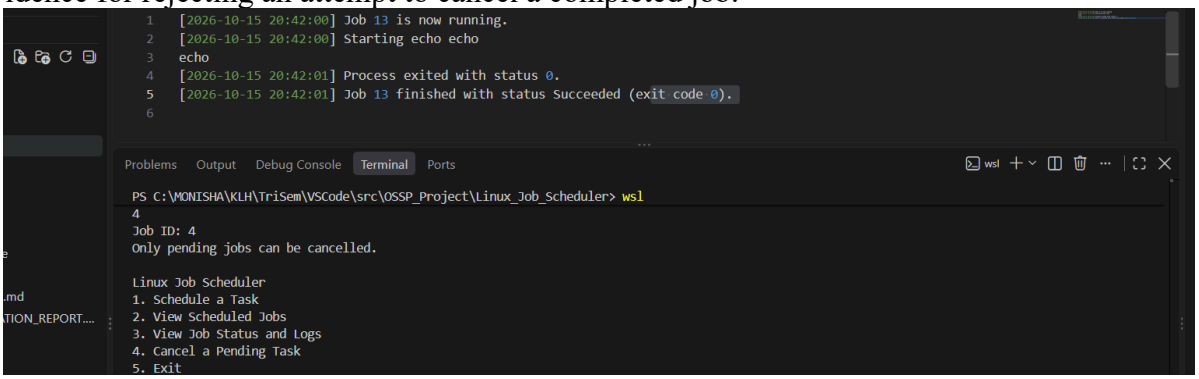



*Figure 7.10: Attempt to cancel a completed job is rejected.*

## 7.7 Failure Handling

The project includes a supported false task that can be used to demonstrate a non-zero exit status. The screenshot below shows the false task being scheduled; refer to the actual execution record or log when discussing its final result.

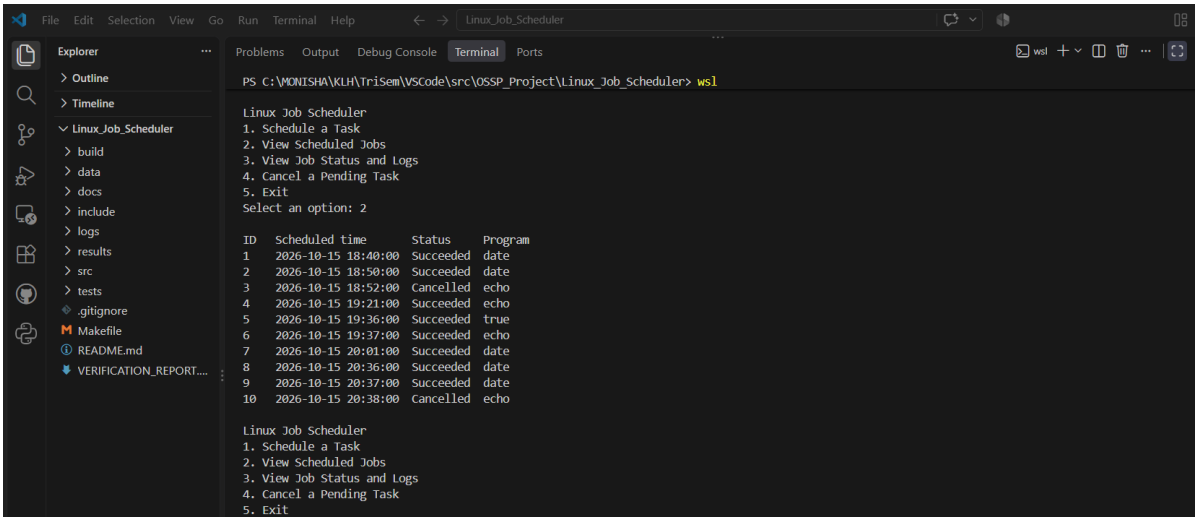



*Figure 7.11: Scheduling a false task to test failure-status handling.*

A successful process typically returns exit code 0, while false returns a non-zero exit code. The report should describe the scheduler's final status only where the supplied execution evidence confirms it.

## 7.8 Demonstration of Operating System Concepts

The Linux Job Scheduler and Task Automation System demonstrates selected Operating System concepts through task scheduling, process execution, file handling, and job-status management.

| S. No. | Operating System Concept | Demonstration in the Project |
|---|---|---|
| 1 | Process Management | Executes supported programs as scheduled tasks and records their execution results. |
| 2 | Task Scheduling | Checks scheduled times and identifies pending jobs that are ready for execution. |
| 3 | Process Exit Status | Uses the program's exit status to distinguish successful execution from failure. |
| 4 | File Handling | Stores job information and execution records in files for later reference. |
| 5 | Data Persistence | Retains job records in storage so that they can be reviewed beyond an individual menu operation. |
| 6 | Input Validation | Validates program selection and scheduled time, rejecting unsupported programs and past times where implemented. |
| 7 | Error Handling | Records task results and helps identify whether a job completed successfully or failed. |
| 8 | Process Communication | Process-related system calls and pipes, if used in the source code, demonstrate process execution and collection of output or results. |
| 9 | Thread Management | If POSIX threads are used, thread creation and joining demonstrate basic thread-management operations. |
| 10 | Security | Restricts execution to supported programs, reducing the risk of unintended command execution. |

## 7.9 Known Limitations

The scheduler must remain running to check and dispatch jobs. It is a terminal-based application, and the set of executable programs is restricted by the implementation. Advanced facilities such as recurring schedules, job dependencies, a graphical interface, and execution while the application is closed should not be claimed unless they have been implemented and tested.

## 7.10 Summary

The supplied execution evidence demonstrates scheduling supported programs, viewing pending jobs, observing execution output and logs, validating input, and cancelling pending jobs. Together, these examples show the main workflow of the Linux Job Scheduler and Task Automation System. The screenshots in this chapter are drawn from the results/outputs directory included in the supplied OSSP Hackathon Submission Package ZIP. They should be retained only if they correspond to the version of the source code submitted for evaluation.

# CHAPTER - 8 CONCLUSION

## 8.1 Summary of the Project

The Linux Job Scheduler and Task Automation System is a C-based application developed to schedule and execute supported programs automatically at specified times. The project provides a terminal-based interface through which users can schedule tasks, view scheduled jobs, check job status and logs, cancel pending tasks, and exit the application.

The system is organized into separate modules for user interaction, job management, scheduling, process execution, and job storage. This modular structure makes the program easier to understand, maintain, and test.

The project demonstrates the practical application of selected Operating System concepts, including process management, time-based task scheduling, file handling, input validation, and execution-status checking. It was developed using C, GCC, Ubuntu, and Visual Studio Code.

## 8.2 Objectives Achieved

The main objective of the project was to develop a system that allows users to schedule programs or tasks for automatic execution at specified times. The project addresses this objective through the following features.

### 8.2.1 Terminal-Based User Interface

A menu-driven terminal interface was developed to allow users to interact with the scheduler. The menu provides options to schedule a task, view scheduled jobs, check job status and logs, cancel a pending task, and exit the application.

### 8.2.2 Task Scheduling

The application allows users to schedule supported programs for a specified future date and time. The scheduler checks pending jobs and identifies tasks that are due for execution while the application is running.

### 8.2.3 Integration of Operating System Concepts (CO1–CO6)

The project applies selected Operating System concepts through its implementation of task scheduling, process execution, file handling, and job management. These features provide practical learning related to the course outcomes. However, the project should not be considered a complete implementation of all six course outcomes, particularly virtual-memory management and advanced thread synchronization, unless these are separately demonstrated.

### 8.2.4 Process Management

The system executes supported programs and checks their execution results. The exit status helps distinguish successful execution from failure. This demonstrates the use of operating-system-supported process execution in a C application.

### 8.2.5 Dynamic Memory Management

The project is primarily focused on scheduling and executing tasks rather than implementing a complete memory-management system. Any dynamic memory allocation demonstrated in the source code or a separate memory-management example provides additional practical exposure to C memory handling. It does not, by itself, demonstrate virtual-memory paging or page replacement.

### 8.2.6 File Storage

The application stores job information and execution records in files, allowing job details and results to be reviewed later. This demonstrates file handling and data persistence in a Linux environment.

### 8.2.7 Thread Management

Where POSIX thread operations are used in the implementation, they demonstrate basic thread creation and completion. The extent of concurrency depends on the actual scheduling logic, and thread creation alone does not prove simultaneous execution of multiple jobs.

### 8.3 Results Achieved

The project provides a working structure for managing scheduled tasks through a terminal interface. Its main results include:

1. Scheduling supported programs for specified future times.
2. Viewing scheduled jobs and their current statuses.
3. Executing tasks when their scheduled times are reached while the application is running.
4. Recording execution results and exit statuses.
5. Viewing available job information and execution logs.
6. Validating program selection and scheduled time.
7. Cancelling pending jobs, where supported by the implementation.

The supplied project screenshots and test evidence can be used to demonstrate these features. The final results should be reported according to the actual behavior observed during testing.

## 8.4 Skills and Learning Outcomes

The development of this project provided practical experience in C programming, Linux development tools, modular program design, task scheduling, process execution, and file handling.

### 8.4.1 Linux Environment and Development Tools

Experience was gained in using Ubuntu, the Linux terminal, GCC, Visual Studio Code, and Makefile-based compilation. These tools supported the development, execution, and testing of the application.

### 8.4.2 C Modular Programming

The project uses separate modules to organize its functionality. This helped demonstrate how a larger C application can be divided into manageable components with distinct responsibilities.

### 8.4.3 System Calls and APIs

The project provides practical exposure to operating-system interfaces used for process execution, file operations, and other supported system functions. The exact system calls and APIs included in the final report should match those present in the submitted source code.

### 8.4.4 Time-Based Scheduling

The project helped develop an understanding of checking scheduled times, identifying pending tasks, and managing their execution status. It also highlighted the difference between application-level task scheduling and kernel-level CPU scheduling.

### 8.4.5 Process Management

The implementation demonstrates how supported programs can be executed and how their results can be checked. This provides practical learning about process execution and exit-status handling.

### 8.4.6 Memory and File Management

The project provided experience in storing job information and maintaining execution records. Any dynamic memory-management concepts should be explained using the relevant implementation or separate demonstration included in the project.

### 8.4.7 Thread Management and Synchronization

If POSIX threads are included, the project provides experience with basic thread creation and completion. Advanced synchronization concepts, such as mutexes, semaphores, and condition variables, should be listed as achieved learning outcomes only if they were implemented and tested.

## 8.5 Overall Conclusion

The Linux Job Scheduler and Task Automation System demonstrates how a C application can be developed to schedule and manage supported programs in a Linux environment. It combines a terminal-based interface with job management, time checking, process execution, and persistent storage to provide a practical task-automation solution.

The project helped connect theoretical Operating System concepts with practical programming techniques. It also provided experience in modular C development, Linux tools, testing, input validation, and execution-result handling.

Although the system has limitations, such as requiring the application to remain running for scheduled jobs to be checked, it provides a foundation for future improvements. These may include recurring schedules, background execution, notifications, improved concurrency, and a graphical user interface.

In conclusion, the project achieves its main purpose of demonstrating a basic Linux-based job scheduling and task automation system while providing practical exposure to selected Operating System concepts.

# CHAPTER - 9 REFERENCES

## 9.1 Books


1. Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, *Operating System Concepts*, Wiley.
2. Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, 2018. Available online at: https://pages.cs.wisc.edu/~remzi/OSTEP/


## 9.2 Research Papers


1. R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, educational materials on operating-system virtualization, concurrency, and persistence, *Operating Systems: Three Easy Pieces*.
2. Additional research papers, if used, should be cited with their actual title, author names, publication, year, and DOI or URL.


## 9.3 Linux / GNU Documentation

### 9.3.1 GCC Documentation


GNU Project, *GCC Online Documentation*. Available at: https://gcc.gnu.org/onlinedocs/


This documentation was used as a reference for compiling and building C programs using GCC.

### 9.3.2 Linux Process Documentation


Linux man-pages project, documentation for process-related system calls, including fork(), exec(), and waitpid(). Available at: https://man7.org/linux/man-pages/


These references are relevant to creating processes, executing programs, and collecting process exit status where the corresponding functions are used in the project.

### 9.3.3 Linux Signal and Alarm Documentation


Linux man-pages project, signal and timer documentation. Available at: https://man7.org/linux/man-pages/


This documentation is relevant only if signal or timer APIs are used in the submitted source code.

### 9.3.4 Linux File I/O Documentation


Linux man-pages project, file I/O documentation for functions and system calls such as open(), read(), write(), and close(). Available at: https://man7.org/linux/man-pages/


These references support the study of file operations used for storing job information and execution records, where applicable.

### 9.3.5 POSIX Thread Documentation

The Open Group, *POSIX Threads (pthreads) Documentation*. Available at: https://pubs.opengroup.org/onlinepubs/9699919799/

This documentation provides reference information about POSIX thread functions and related interfaces, if they are used in the implementation.

## 9.4 Project Resources

### 9.4.1 Project Source Repository

Linux Job Scheduler and Task Automation System, GitHub repository. Available at: https://github.com/monishachelle-max/KLH_CSE_2026_27_16_Linux-Job-Scheduler-and-Task-Automation-System.git

### 9.4.2 Project README and Source Files

The project's README file, C source files, header files, Makefile, and supporting test files were used to understand the project structure and implementation. These files are available in the project repository.

### 9.4.3 Project Execution Screenshots

The execution screenshots included in the project report provide visual evidence of application compilation, task scheduling, job-status display, execution results, log viewing, input validation, and pending-job cancellation, where those operations are shown in the supplied project evidence.

## 9.5 Reference Usage in the Project

The references listed in this chapter provide background information about Operating System concepts and the tools used to develop the Linux Job Scheduler and Task Automation System. GCC documentation supports the study of C program compilation, while Linux manual pages provide information about process management and file operations. POSIX documentation provides reference material for thread-related functionality where applicable.
The project's source repository and README provide information about the actual implementation, module organization, build process, and available functionality.

## 9.6 Summary of References

This chapter lists the books, educational materials, official technical documentation, and project resources relevant to the development of the Linux Job Scheduler and Task Automation System. These references support the understanding of C programming, Linux process execution, file handling, task scheduling, and other selected Operating System concepts.